

“Encrypted Secret File Transfer Distributed Backend Database System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Y.EESHA YUKTHA

2520030436

P. SRESHTA

2520030217

V. SREE PRAGNYA

2520030238

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Encrypted Secret File Transfer Distributed Backend Database System”** is a record of Bonafide work of **Y.EESHA YUKTHA -2520030436, P. SRESHTA - 2520030217, V. SREE PRAGNYA - 2520030238** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Y. EESHA YUKTHA – 2520030436

V SREE PRAGNYA – 2520030238

P.SRESHTA–2520030217

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“Encrypted Secret File Transfer Distributed Backend Database System”** is a bonafide work done and submitted by **Y.EESHA YUKTHA -2520030436, P. SRESHTA - 2520030217, V. SREE PRAGNYA - 2520030238** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Yerragudipadu Subbarayudu**, and my guide **Dr Yerragudipadu Subbarayudu**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Y. EESHA YUKTHA – 2520030436

V SREE PRAGNYA – 2520030238

P.SRESHTA–2520030217

Abstract

The Encrypted Secret File Transfer Distributed Backend Database System is a secure web-based application developed to provide protected file storage, controlled file sharing, and secure transfer of sensitive files between authorized users. The main objective of the system is to overcome security and privacy concerns associated with conventional file-sharing methods by providing authentication, authorization, encryption, and controlled access to files.

The system allows users to register and securely log in to the application before accessing the available file management features. Authenticated users can upload files, view their stored files, manage file permissions, share files with authorized users, and download files based on the permissions assigned to them. The system maintains information related to users, files, permissions, and file-transfer activities, enabling controlled and traceable file management.

The application is developed using React.js for designing the interactive frontend and Node.js with Express.js for implementing the backend services and RESTful APIs. MongoDB is used as the database for storing user, file, permission, and transfer-related information, with Mongoose providing structured interaction between the Node.js application and MongoDB. JWT (JSON Web Token) is used for secure user authentication and authorization, while bcrypt is used for securely hashing user passwords. Node.js cryptographic features are incorporated to provide encryption and protection for sensitive files and data during the file-transfer process.

The system follows a distributed backend approach in which the frontend communicates with backend APIs for authentication, file management, permission management, and transfer operations. Appropriate validation and access-control mechanisms are implemented to ensure that users can perform only the operations for which they have permission. The system also maintains relevant file and transfer information in the database, supporting better monitoring and management of file activities.

The proposed system provides a user-friendly and secure environment for handling confidential files. By combining authentication, authorization, password hashing, encryption, database management, and controlled file sharing, the system aims to improve the confidentiality, integrity, and controlled accessibility of sensitive information. The project demonstrates how modern web technologies and backend security mechanisms can be integrated to develop a reliable platform for secure file storage and transfer.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9-16
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	17-19
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	20-27
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	28-34
5	Design	5.2 ER Diagram / Database schema	35-40
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	41-48
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	49-51
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	52-55
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	56-58
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	59-61
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	62-63
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	64-65
13	References	- Books, tutorials, APIs, documentation sites used	66

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	67-82
----	------------	---	-------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Digital file sharing has become an essential part of everyday academic, professional, and personal communication. Individuals and organizations routinely exchange documents, images, reports, and other digital assets over the internet using email attachments, cloud-storage links, messaging applications, and third-party file-sharing services. As the volume of digital communication has grown, so has the sensitivity of the information being exchanged; contracts, personal identification documents, financial records, and confidential project reports are now commonly transferred through channels that were never specifically designed with strong security guarantees in mind.

Ordinary file-transfer mechanisms typically move data from one point to another without adequately verifying who is permitted to access the file, and without protecting the content of the file itself while it is stored or in transit. When a file is transmitted or stored without encryption, it becomes vulnerable to interception on an insecure network, unauthorized access on a compromised server, or accidental exposure through a misconfigured storage location. Even when a transport-layer protocol such as HTTPS is used to secure communication between a browser and a server, the file may still be stored on the server in plain form, meaning that anyone who gains access to the storage medium can read its contents directly.

These risks are amplified in scenarios involving confidential or “secret” files – documents that must be seen only by specific, authorized individuals. Data interception during transmission, unauthorized access to stored files, and data leakage due to weak or absent access control are recurring concerns in real-world systems. Addressing them requires a combination of techniques rather than a single safeguard: authentication to confirm the identity of each user, authorization to determine what an authenticated user is permitted to do, encryption to protect the file's content, and a reliable database to keep track of users, files, permissions, and activity so that the system's behaviour can be audited and reasoned about.

The growth of backend development frameworks and NoSQL databases has made it considerably easier for student developers to design and build such systems using modern, well-documented, and freely available tools. Node.js and Express.js provide a lightweight yet powerful environment for building RESTful APIs; React.js enables the construction of responsive, component-based user interfaces; and MongoDB, together with the Mongoose ODM,

offers a flexible document-oriented data model that is well suited to storing users, file metadata, permissions, and transfer logs without the rigid schema constraints of a traditional relational database.

The Encrypted Secret File Transfer Distributed Backend Database System is motivated by this background. It is conceived as a modular, database-driven web application in which distinct backend responsibilities – authentication, file handling, encryption, and permission management – are organized into separate logical modules that communicate through REST APIs. This design choice is directly relevant to the subjects of Database Systems Engineering and Distributed Backend Development, since it requires careful schema design for the underlying MongoDB collections, thoughtful API design between the frontend and backend, and an architecture that can, in principle, be decomposed further into independently deployable services as the system scales.

From an academic perspective, the project offers hands-on exposure to full-stack web development, secure authentication design, symmetric file encryption, NoSQL database modelling, and RESTful API testing using Postman. It is scoped to be achievable within a single academic semester by a team of three second-year students while still being technically substantial enough to demonstrate the core competencies expected of a database-systems and backend-development project.

Relevance to Database Systems Engineering

Database Systems Engineering is concerned with how data is modelled, stored, queried, protected, and kept consistent. This project touches each of these concerns. The team must decide which entities the system needs (users, files, permissions, and transfer logs), how these entities relate to one another, which fields must be unique or indexed, and how to keep the metadata stored in the database consistent with the encrypted files kept in file storage. Decisions such as storing a reference to the owner instead of copying the owner's details into every file document are typical design questions in this subject.

Relevance to Distributed Backend Development

Distributed Backend Development is concerned with structuring server-side systems into cooperating components that communicate through well-defined interfaces. Even though this project runs as a single Node.js service, its authentication, file handling, encryption, permission, and logging responsibilities are separated into modules with clear boundaries, and the frontend interacts with them only through REST APIs. This gives the team practical experience with the design habits – statelessness, clear contracts, and separation of concerns – that make it possible

to distribute a backend across several services later.

1.2 Problem Statement

Traditional file-transfer systems used by many small teams and individuals – including generic cloud-storage links, plain email attachments, and ad-hoc file servers – do not consistently guarantee that a confidential file remains protected from unauthorized users, either while it is stored on a server or while it is being transferred between the sender and the intended recipient. In many such systems, any user who can guess or obtain a link, or who gains access to the underlying storage, can read the file's contents directly, because the file itself is never transformed into an unreadable form.

In addition, many lightweight file-sharing tools lack a proper authentication and authorization layer: they either allow anonymous access altogether, or they authenticate a user only loosely without enforcing per-file access permissions. As a result, there is no reliable way to specify that a particular file may be accessed only by a specific set of authorized users, nor is there a dependable mechanism to track who accessed, uploaded, or downloaded a given file and when.

This project addresses the problem by designing a file-transfer system built around four complementary safeguards: (1) user authentication using JWT-based tokens so that only verified users can interact with the system, (2) file encryption so that the content of an uploaded file is unreadable to anyone who does not possess the appropriate decryption capability, (3) a permission-based authorization model so that a file owner can explicitly control which users may access a given file, and (4) a MongoDB-backed database that stores user accounts, file metadata, permissions, and a transfer log, giving the system a persistent, queryable, and auditable record of its own activity.

The problem statement can therefore be summarised as follows: to design and document a secure, database-driven file transfer system in which confidential files are encrypted before storage, users are authenticated and authorized before they can upload, share, or download files, and every significant file-related action is logged, all implemented using a React.js frontend, a Node.js/Express.js backend, and a MongoDB database accessed through Mongoose.

Why Each Safeguard Is Needed

- Encryption is needed because a file that is stored in readable form is exposed to anyone who reaches the storage medium, whether through a stolen backup, a misconfigured server, or a compromised account.

- Authentication is needed because the system must know who is making each request before deciding what to allow.
- Authorization is needed because being a valid user does not mean being entitled to every file; each file should be accessible only to its owner and the users the owner has approved.
- Secure file management is needed so that files are stored under controlled names and locations and are not directly reachable from the web.
- Database storage is needed to keep a persistent, queryable record of users, files, and permissions.
- Transfer logging is needed so that uploads, downloads, and permission changes can be reviewed later.

1.3 Scope of the Project

The scope of this project covers the design and proposed implementation of a web-based, encrypted file-transfer system intended for use by a defined group of authenticated users, such as members of a team, a class, or an organization, rather than for anonymous public file sharing. Within this scope, the system is expected to support user registration and login, secure file upload with encryption, storage of file metadata in MongoDB, permission-based file sharing between registered users, secure file download with decryption for authorized users, and a transfer log that records upload, download, and permission-change events.

The system's backend is scoped to a modular Node.js/Express.js application in which authentication, file handling, encryption, and permission management are implemented as distinct logical modules, communicating through REST APIs and documented and testable in Postman. The frontend is scoped to a React.js single-page application providing registration, login, a file dashboard, an upload interface, a permission-management interface, and a download interface.

The following aspects are explicitly outside the scope of the present project: integration with third-party cloud storage providers, mobile native applications, real-time chat or notification systems, virus or malware scanning of uploaded files, and large-scale, geographically distributed deployment across multiple physical servers. These items are identified in Chapter 11 as potential future enhancements rather than as part of the current implementation. Similarly, the project does not claim to provide military-grade or certified cryptographic assurance; it instead demonstrates sound, widely used encryption and authentication practices appropriate for an

academic, second-year B.Tech project.

The scope is intentionally bounded so that the system remains achievable within one academic semester by a three-member student team, while still being comprehensive enough to demonstrate meaningful competency in database design, backend API development, authentication and authorization, and file encryption – the principal learning outcomes targeted by this project.

Table 1.1 – Scope Summary

Included in Scope	Not Included in Scope
User registration and login	Anonymous public file sharing
JWT authentication and bcrypt password hashing	Third-party single sign-on providers
File upload with validation and encryption	Malware or virus scanning
File metadata storage in MongoDB	Storing entire file binaries inside MongoDB
Owner-managed permissions and authorized download	Enterprise-scale multi-region deployment
Transfer logging	Real-time notifications and mobile applications
REST API testing with Postman	Formal security certification or audit

1.4 Objectives of the Project

The primary objective of the project is to design and document a secure, database-driven file transfer system that protects confidential files from unauthorized access while remaining simple enough for a second-year B.Tech team to implement and demonstrate. This overarching objective can be broken down into the following specific objectives:

- To implement secure user registration and login using JWT-based authentication, ensuring that only verified users can access the system's features.
- To design and apply a password-hashing mechanism using bcrypt so that user credentials are never stored or compared in plain text.
- To design a file encryption and decryption workflow so that uploaded files are stored in an encrypted form and can only be restored to a readable form through an authorized process.
- To design a MongoDB database schema – comprising users, files, permissions, and transfer_logs collections – that can reliably represent accounts, file metadata, sharing permissions, and activity history.

- To design a permission-management mechanism allowing file owners to grant or revoke access to specific files for specific users, and to enforce these permissions on every download request.
- To develop RESTful APIs using Node.js and Express.js for all core operations – registration, login, upload, permission management, download, and logging – and to verify these APIs using Postman.
- To develop a React.js frontend that allows users to interact with the above features through a clear, usable interface.
- To maintain a transfer log that records file-related activities such as uploads, downloads, and permission changes for accountability and traceability.
- To demonstrate, at a conceptual level, how the backend's responsibilities can be organized into modular components consistent with the principles of distributed backend architecture, and to describe how these modules could be decomposed into independently deployable services in a future iteration of the system.

1.5 Importance of the Application

From a security perspective, the application is important because it demonstrates, in a concrete and hands-on manner, how authentication, authorization, and encryption work together to protect sensitive data – concepts that are frequently taught in isolation but rarely combined into a single working system at the undergraduate level. By building a system in which files are meaningfully protected rather than merely stored, the project reinforces an appreciation of defense-in-depth: no single safeguard, whether it is a login form or a single encryption step, is treated as sufficient on its own.

From a database systems perspective, the project is important because it requires careful modelling of related entities – users, files, permissions, and transfer logs – within a NoSQL, document-oriented database. Designing these collections, deciding which fields to store as references versus embedded data, and ensuring that permission checks can be performed efficiently are all practical exercises in database design that extend beyond simple CRUD operations.

From a backend development perspective, the project is important because it requires structuring a non-trivial set of REST APIs around authentication middleware, file-handling routes, and permission checks, giving the team direct experience with the kind of modular backend

organization used in production-grade Node.js/Express.js applications. From a user perspective, the system is important because it results in a working application that meaningfully reduces the risk of a confidential file being read by someone who was never meant to see it, addressing a genuine, relatable problem rather than a purely theoretical one.

Finally, from an academic perspective, the project is valuable because it satisfies the course outcomes associated with Database Systems Engineering and Distributed Backend Development by requiring the team to apply schema design, REST API development, authentication, encryption, and testing within a single, coherent project, and to present and defend these design decisions during project review and viva.

Table 1.2 – Importance by Perspective

Perspective	Why the Project Matters
Security	Combines authentication, authorization, and encryption as layered defences
Database	Applies NoSQL schema design, references, and indexing to a realistic problem
Backend development	Practises REST API design, middleware, and modular structure
User	Reduces the chance that a confidential file is read by the wrong person
Academic	Integrates course outcomes in a single reviewable project

1.6 Target Users / Audience

The system is designed with several categories of potential users in mind, even though the current implementation is scoped as an academic project rather than a production deployment. Students and faculty members represent a natural target audience, since academic environments frequently require the confidential exchange of documents such as assignments, question papers, evaluation sheets, and research drafts among specific, authorized individuals.

Small teams and organizations that need to share confidential internal documents – such as project reports, internal memos, or draft contracts – among a defined set of members represent another target audience. Employees and administrators within an organization who require controlled access to sensitive files, with the ability to grant or revoke access as team composition changes, are also well served by the permission-management features of the system.

More broadly, any user who needs to transfer confidential documents to a specific recipient, and who wants assurance that the file cannot be read by anyone else even if the storage or network layer is compromised, is a target user of this system. The system is not designed for anonymous public file sharing or for use cases requiring extremely high-throughput file transfer at an

enterprise scale; it is instead intended to demonstrate, for a bounded and identifiable set of users, how encryption, authentication, and access control can be combined into a coherent and usable secure file-transfer experience.

Table 1.3 – Target Users and Typical Needs

User Group	Typical Need
Students	Share confidential project drafts and documents within a team
Faculty	Exchange question papers, evaluation sheets, and research material
Organizations and teams	Distribute internal documents to specific members
Employees	Send sensitive documents to authorized colleagues
Administrators	Manage users and review activity records
Any user of confidential documents	Transfer a file so that only the intended recipient can read it

CHAPTER 2 – SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Because the proposed system is a web application built on Node.js and MongoDB, its hardware requirements are modest and comparable to those of any typical full-stack JavaScript development environment. The requirements below distinguish between the minimum configuration needed to develop and run the system locally and a recommended configuration that provides a smoother development experience, particularly when running the React development server, the Node.js backend, and a local MongoDB instance simultaneously on the same machine.

Table 2.1 – Hardware Requirements

Component	Minimum Requirement	Recommended Requirement	Purpose
Processor	Dual-core, 2.0 GHz	Quad-core, 2.5 GHz or higher	Running the Node.js server, React build tools, and MongoDB concurrently
RAM	4 GB	8 GB or higher	Supporting simultaneous frontend, backend, and database processes
Storage	20 GB free disk space	50 GB free disk space (SSD preferred)	Storing source code, dependencies (node_modules), and encrypted files
Internet Connection	Basic broadband	Stable broadband (for package installation and API testing)	Downloading npm packages and testing deployed endpoints
Display	1366 x 768 resolution	1920 x 1080 resolution	Comfortable viewing of code editors, browser, and Postman side by side
Keyboard / Mouse	Standard keyboard and mouse	Standard keyboard and mouse	General development interaction

These requirements reflect a typical student laptop or desktop configuration and do not assume access to specialized server hardware, GPUs, or high-performance computing resources, since the system's computational load – authentication, file encryption of moderate-sized documents, and standard REST API request handling – is not resource-intensive at the scale targeted by this project.

2.2 Software Requirements

The software requirements of the system are centred on the JavaScript ecosystem, reflecting the choice of React.js for the frontend and Node.js with Express.js for the backend, together with MongoDB as the database and a small set of supporting development tools.

Table 2.2 – Software Requirements

Software	Purpose
Windows / Linux / macOS	Host operating system for development
Node.js (LTS version)	JavaScript runtime for running the backend server and frontend build tools
npm	Package manager used to install and manage project dependencies
React.js	Frontend JavaScript library used to build the user interface
Express.js	Backend web framework used to build REST APIs on top of Node.js
MongoDB	NoSQL document database used to store users, file metadata, permissions, and logs
MongoDB Compass	GUI tool used to inspect and manage MongoDB collections during development
Mongoose	Object Data Modelling (ODM) library used to define schemas and interact with MongoDB from Node.js
Visual Studio Code	Code editor used for writing and debugging frontend and backend code
Postman	Tool used to test and document REST API endpoints
Git	Version control system used to track changes to the source code
GitHub	Remote repository hosting platform used for collaboration among team members
Modern Web Browser (Chrome/Edge/Firefox)	Used to run and test the React frontend application

Node.js provides the runtime in which both the Express.js backend and the React build tools execute, while npm installs the libraries used by both. Mongoose depends on a running MongoDB server, which may be a local installation or a cloud-hosted instance. The versions of these tools should be recorded by the team (for example, in package.json and in the installation appendix) so that the environment can be reproduced on another machine.

Postman and MongoDB Compass are used only during development and testing. They are not part of the deployed system, but they are essential for verifying that the API and the database behave as designed.

2.3 Development Tools and Frameworks

In addition to the core software listed above, the development process relies on a small set of supporting tools and conventions that streamline collaboration among the three team members and help maintain code quality throughout the project.

Visual Studio Code is used as the primary code editor for both the frontend and backend, taking advantage of extensions for JavaScript/React syntax highlighting, ESLint-based linting, and integrated terminal access for running npm scripts. The project follows a conventional folder structure separating the client (React) and server (Node.js/Express) codebases, which keeps frontend and backend concerns cleanly separated even though both are developed within the same overall repository.

Git and GitHub are used for version control and collaboration: each team member works on feature branches corresponding to a specific module (for example, authentication, file handling, or permission management), and changes are merged into a main branch after review. This workflow mirrors standard industry practice for small development teams and gives every team member a clear, traceable history of who implemented which part of the system.

Postman is used throughout development to design, test, and document the REST API endpoints exposed by the Express.js backend before or alongside frontend integration. This allows the backend team members to verify that authentication, file upload, permission management, and download endpoints behave correctly using representative requests, independent of the React frontend. MongoDB Compass is used to visually inspect the users, files, permissions, and transfer_logs collections during development, helping the team confirm that data is being written and structured as intended by the Mongoose schemas.

CHAPTER 3 – TECHNOLOGY STACK

3.1 Front-End Design

The frontend of the Encrypted Secret File Transfer Distributed Backend Database System is built using React.js, a component-based JavaScript library for constructing interactive user interfaces. React.js is used in the proposed system to create the interactive frontend through which users can register, log in, upload files, view file metadata, manage access permissions, and request authorized downloads.

The frontend is organized as a set of reusable React components, each responsible for a specific piece of the interface: a registration form component, a login form component, a file-upload component, a file-listing/dashboard component, a permission-management component, and a download component. Each component manages its own local state using React's `useState` hook where appropriate – for example, tracking the currently selected file, the values entered into a form, or whether a request is currently in progress – while data that needs to be shared across components, such as the currently authenticated user and their JWT token, is managed using React's Context API or lifted to a common parent component.

React Router is used to implement client-side navigation between the login page, registration page, dashboard, and other views, allowing the application to behave as a single-page application (SPA) that updates its content without full page reloads. Forms in the application – for registration, login, and file upload – are built as controlled components, where the value of each input field is bound to React state and updated through `onChange` event handlers, allowing the application to validate input (for example, checking that an email field looks well-formed, or that a password meets a minimum length) before submitting a request to the backend.

API calls from the frontend to the backend are made using the Fetch API or a lightweight HTTP client such as Axios, wrapped in helper functions that automatically attach the JWT token (once the user has logged in) to the Authorization header of protected requests. Error handling is implemented so that failed requests – for example, an incorrect password during login, or an unauthorized download attempt – result in clear, user-facing error messages rather than silent failures.

It is important to state clearly that Node.js and npm are not part of the frontend itself; rather, they provide the development and runtime environment required to build and serve the React application. Node.js is used to run the local development server during development and to

execute the build process that compiles the React application into static HTML, CSS, and JavaScript files for deployment, while npm manages the JavaScript packages (React, React Router, Axios, and related libraries) that the frontend depends upon.

The user interface itself is designed to be responsive, ensuring that the registration form, login form, file dashboard, and upload/download interfaces remain usable on both desktop and smaller screen sizes. Basic accessibility considerations, such as labelled form fields and clear error and success messages, are incorporated so that the interface remains understandable to a broad range of users.

Table 3.1 – Technology Stack

Layer	Technology	Role in the Project
Frontend	React.js, JavaScript, HTML, CSS	User interface for registration, login, upload, sharing, and download
Frontend environment	Node.js and npm	Development server, build tooling, and package management for the React application
Backend	Node.js and Express.js	REST APIs, middleware, encryption and permission logic
Database	MongoDB	Persistent storage of users, file metadata, permissions, and logs
Database interaction	Mongoose	Schemas and queries from the Node.js backend to MongoDB
API testing	Postman	Testing and documenting REST endpoints
Version control	Git and GitHub	Change tracking and collaboration
Development tools	Visual Studio Code, MongoDB Compass, npm	Editing code, inspecting data, managing packages
Security	JWT, bcrypt, file encryption, input validation, environment variables	Authentication, credential protection, content protection, safe configuration

Key React Concepts Used

- Components: the interface is divided into small reusable units such as LoginForm, UploadForm, FileList, and PermissionPanel.
- Props: parent components pass data such as a file record to child components that display it.
- State: values that change over time, such as form input or the list of files, are held with the useState hook.
- Hooks: useEffect loads the file list when the dashboard opens, and useContext shares the authenticated user.

- Forms and event handling: controlled inputs and onSubmit handlers gather data and call the API service.
- React Router: maps URLs such as /login and /dashboard to components and supports protected routes.
- Error handling: failed requests set an error state that is displayed to the user.

3.2 Back-End Design

The backend of the system is built using Node.js as the JavaScript runtime and Express.js as the web framework that structures the application's REST APIs. Node.js and Express.js together form the backend layer of the system, distinct from MongoDB, which serves purely as the database.

The Express.js application is organized around a set of route files, each grouping related endpoints: authentication routes (registration and login), file routes (upload, listing, and download), and permission routes (granting and revoking access). Each route delegates its core logic to a corresponding controller function, keeping the routing layer thin and the business logic centralized and testable. Middleware functions are used extensively throughout the backend: a JWT-verification middleware checks the Authorization header on protected routes and rejects requests with missing or invalid tokens before they reach the controller logic; a file-validation middleware, built on top of a library such as Multer, checks the type and size of an uploaded file before it is passed to the encryption module; and a centralized error-handling middleware ensures that errors from any layer of the application are converted into consistent, well-structured JSON error responses.

Node.js is well suited to handling file-related requests and API communication in this system because of its non-blocking, event-driven I/O model, which allows the server to continue handling other incoming requests (such as authentication checks for other users) while a potentially slow operation – such as reading a large file from disk for encryption, or writing an encrypted file back to storage – is in progress. This characteristic makes Node.js a practical choice for a system whose primary workload consists of many relatively small, I/O-bound operations: verifying tokens, reading and writing files, and communicating with MongoDB.

Database communication from the backend is handled exclusively through Mongoose, which is described in detail in Section 3.3 below. The backend also performs input validation on all incoming requests – checking, for example, that an email field is present and well-formed during registration, or that a permission value supplied during a sharing request is one of the

values the system recognises – before any data reaches the database layer, reducing the risk of malformed or malicious data being persisted.

Logging is implemented at the backend level using a simple logging utility (or a library such as Morgan for HTTP request logging) so that the team can trace the flow of requests through the system during development and debugging, complementing the `transfer_logs` collection in MongoDB, which records application-level events such as uploads, downloads, and permission changes rather than raw HTTP traffic.

Table 3.2 – Backend Building Blocks

Concept	Description	Example in this Project
Route	Maps an HTTP method and URL to a handler	POST <code>/api/auth/login</code>
Controller	Contains the logic for a route	<code>login</code> , <code>uploadFile</code> , <code>downloadFile</code>
Middleware	Runs before controllers to check or transform requests	JWT verification, upload validation
Model	Mongoose schema and methods for a collection	User, File, Permission, TransferLog
Utility	Reusable helper functions	<code>encrypt</code> , <code>decrypt</code> , <code>logger</code>

Separating routes, controllers, models, and utilities keeps each file small and testable and makes it easy to see which module is responsible for a given behaviour. It also mirrors the logical modules shown in the architecture diagram.

3.3 Database Design

MongoDB is a NoSQL, document-oriented database used in this project to store all persistent application data: users, files, permissions, and transfer logs. Unlike a relational database, MongoDB does not organize data into tables of rows and columns; instead, data is organized into collections of documents, where each document is a self-describing JSON-like structure (stored internally in a binary form called BSON) that can have a flexible schema.

Each document in a MongoDB collection is uniquely identified by an `_id` field, which MongoDB represents using the `ObjectId` type by default. This project uses four principal collections – users, files, permissions, and `transfer_logs` – described in detail in Chapter 5 (Design). Relationships between these collections, such as the association between a file and its owner, are represented using references: a document in the files collection stores the `ObjectId` of the corresponding user document in its `ownerId` field, rather than embedding the user's full details within the file document itself.

MongoDB's flexible schema is well suited to this project because it allows fields to be added or adjusted as the system evolves – for instance, adding an optional “expiry date” field to a file document in a future enhancement – without requiring a rigid, pre-defined table structure to be altered through a formal schema migration, as would typically be required in a relational database. At the same time, this project applies Mongoose schemas (described in Section 3.4-equivalent detail below) to bring a controlled, application-level structure to each collection, so that the flexibility of MongoDB does not come at the cost of predictable, well-defined document shapes.

Indexing is used on frequently queried fields to keep the system responsive as the number of users and files grows: for example, an index on the email field of the users collection supports fast lookups during login, and indexes on the fileId and userId fields of the permissions and transfer_logs collections support fast permission checks and log queries. MongoDB Compass is used during development to inspect these collections visually, confirm that documents are being created with the expected fields, and manually verify indexes and query performance.

MongoDB Atlas, a cloud-hosted version of MongoDB, is discussed in Chapter 9 (Deployment) as an optional deployment target for the database, offered as an alternative to running MongoDB locally during development; its use is not assumed as part of the core design described in this chapter.

Mongoose

Mongoose is an Object Data Modelling (ODM) library used by the Node.js backend to interact with MongoDB. It lets the team define a schema for each collection, with field types, required flags, defaults, and uniqueness rules, and then use model methods to create, read, update, and delete documents. Because MongoDB itself does not force a schema, Mongoose provides the structure and validation the application needs.

Why MongoDB Is Suitable

- Documents map naturally onto JavaScript objects, so data moves between Node.js and the database with little conversion.
- The flexible schema supports future fields, such as file expiry, without a formal migration.
- Collections for users, files, permissions, and logs are simple to model and query.
- MongoDB Compass gives a visual view of data that is helpful when learning and debugging.

- MongoDB Atlas offers an optional path to cloud hosting.

MongoDB is used here as the database only; the backend is the Node.js and Express.js application. Also, because MongoDB is not a relational database, it stores documents in collections rather than rows in tables, and relationships are expressed through references rather than foreign-key constraints, so the application must take care to keep references consistent.

3.4 Version Control

Git is used as the version control system for this project, allowing the three-member team to track changes to both the frontend and backend codebases over time, and to collaborate without overwriting one another's work. Each significant change – implementing an API endpoint, adding a React component, or adjusting the MongoDB schema – is recorded as a commit with a descriptive message, creating a clear history of how the system evolved from an empty repository to a complete application.

GitHub is used to host the project's remote repository and to coordinate collaboration among team members. Feature branches are created for major pieces of work (for example, a branch for the authentication module and a separate branch for the file-encryption module), allowing team members to work in parallel without interfering with each other's code. Once a feature branch is complete and tested, it is merged into the main branch, ideally after a peer review of the changes by another team member.

This workflow gives the team several practical benefits: it provides a safety net that allows any change to be reverted if it introduces a bug, it provides a transparent record of individual contributions that can be referenced during project evaluation, and it mirrors the collaborative development practices used in professional software engineering environments, giving the team direct, hands-on experience with a skill that extends well beyond this specific project.

Table 3.3 – Git Concepts Used

Concept	Use in the Project
Repository	Holds the client and server code and history
Commit	A saved snapshot with a message describing the change
Branch	An isolated line of work such as feature/auth or feature/encryption
Merge / pull request	Combines a reviewed branch into the main branch
Version history	Lets the team see what changed, when, and by whom

3.5 APIs / External Services

The system's frontend and backend communicate exclusively through a set of custom REST APIs exposed by the Express.js server; no third-party external API is required for the system's core functionality, which distinguishes this project from applications that depend on external services such as payment gateways or mapping providers.

The overall communication pattern follows a consistent chain: the React frontend issues an HTTP request to a REST API endpoint, which is received by the Express.js backend; the backend's routing and middleware layer authenticates the request and validates its contents; a controller function then performs the requested operation, communicating with MongoDB through Mongoose as needed; and a JSON response – containing either the requested data or a structured error message – is returned to the frontend, which updates the user interface accordingly.

REST conventions are followed throughout the API design: resources are referred to using nouns in the URL path (for example, `/api/files` or `/api/permissions`), and the appropriate HTTP verb is used to indicate the intended operation – POST for creating a new resource (registering a user or uploading a file), GET for retrieving resources (listing a user's files), PUT/PATCH for updating a resource (changing a permission), and DELETE for removing a resource (revoking access or deleting a file record). This consistency makes the API easier to document, test in Postman, and reason about during development.

Postman is used as the primary tool for testing these internal APIs during development, allowing each endpoint to be exercised independently of the React frontend, with representative request bodies, headers (including the Authorization header carrying a JWT), and expected response codes documented in a Postman collection shared among the team. This ensures that backend functionality can be verified in isolation before it is integrated with the frontend, and provides a lightweight form of API documentation that can be referenced by any team member working on either side of the application.

The chain of communication in the system can be written as: React frontend, REST API, Express backend, Mongoose, MongoDB. A request from the browser travels left to right, and the response travels back. Each arrow in this chain is an interface with its own rules: JSON over HTTP between the frontend and backend, and Mongoose queries between the backend and the database.

Table 3.4 – REST Verb Usage

HTTP Verb	Meaning	Example
GET	Read data	List the user's files
POST	Create data or perform an action	Register, login, upload, grant permission
PUT / PATCH	Update data	Change a permission level
DELETE	Remove data	Revoke a permission

CHAPTER 4 – SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The proposed system follows a layered, modular architecture in which a React.js frontend communicates with a Node.js and Express.js backend through REST APIs over HTTP, and the backend in turn coordinates a set of logical modules – authentication, file handling, permission management, and encryption – that interact with MongoDB (through Mongoose) and a secure file-storage location.

Figure 4.1 – High-Level System Architecture

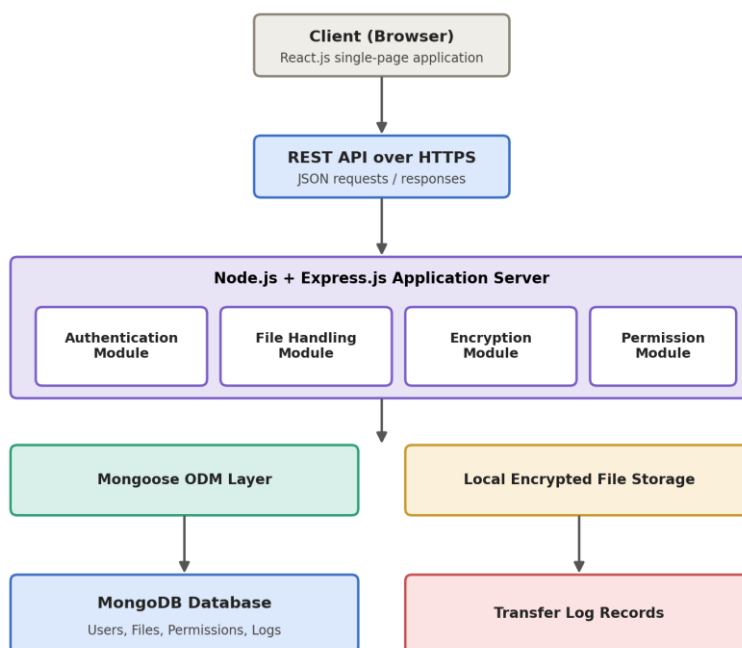


Figure 4.1 – High-Level System Architecture

A user interacts only with the React frontend in a web browser. Every action that requires server-side processing is converted into an HTTP request to the REST API. The Express.js application receives the request, passes it through middleware (JWT verification, input validation), and routes it to the appropriate module. The Authentication Module handles registration and login; the File Module handles upload and download; the Permission Module decides who may access which file; and the Encryption Module encrypts files before storage and decrypts them for authorized downloads.

The database layer stores structured information – users, file metadata, permissions, and transfer logs – in MongoDB, while the file content itself, in its encrypted form, is kept in a separate file-

storage location on the server. This separation between file metadata and file content is a deliberate architectural choice: MongoDB is well suited to storing small, queryable documents, whereas encrypted file content is better held in a storage mechanism designed for files.

It is important to describe the term “distributed backend” accurately. In this project, the modules are logical modules or services within a modular backend; they are not claimed to be deployed on separate physical servers. The design deliberately keeps clear boundaries and communication interfaces between modules so that, in future, each could be extracted into an independently deployed service.

Figure 4.2 — Data Flow Diagram (Level 1)

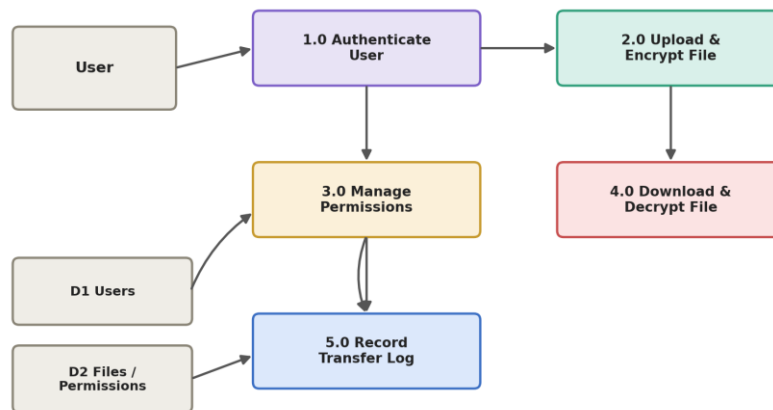


Figure 4.2 – Data Flow Diagram

Figure 4.2 shows the flow of data between the user, the main processes of the system (authentication, upload and encryption, permission management, download and decryption, and transfer logging), and the principal data stores. Authentication precedes every other process; permission checks precede every download; and each significant action results in a transfer log record.

The architecture can be read from top to bottom as a chain of responsibility. The client is responsible only for presentation. The API layer is responsible for accepting well-formed requests. The application server is responsible for enforcing security and business rules. The data layer is responsible for persistence. Keeping these responsibilities apart means a change in one layer, for example replacing the local file storage with cloud storage, has limited effect on the others.

4.2 Description of Each Layer

Frontend Layer

The frontend layer is the React.js application running in the user's browser. It renders the registration, login, dashboard, upload, permission, and download interfaces, holds the user's JWT for the duration of the session, and communicates with the backend exclusively via REST API calls. It performs basic client-side validation for user convenience, but never relies on it for security.

Backend Layer

The backend layer is the Node.js and Express.js application. It exposes the REST API, applies middleware, and coordinates the logical modules. All security-relevant decisions – verifying tokens, checking permissions, encrypting and decrypting files – are made here, because the frontend runs in an environment that the user controls and therefore cannot be trusted.

Authentication Layer

The Authentication Module handles user registration and login. During registration it validates the input, hashes the password using bcrypt, and stores the user in MongoDB. During login it compares the submitted password against the stored hash and, on success, issues a signed JWT. A middleware component verifies this token on every protected route.

File Processing Layer

The File Module receives uploaded files (using multipart/form-data), validates their type and size, passes them to the Encryption Module, stores the resulting encrypted file, and creates the corresponding metadata document. For downloads, it retrieves the encrypted file after a successful permission check.

Encryption Layer

The Encryption Module contains the logic for encrypting and decrypting file content using a symmetric encryption algorithm available in Node.js's built-in crypto module (for example, AES-256 in an authenticated mode such as GCM). The encryption key is supplied through an environment variable and is never stored in source code or in the database.

Database Layer

The database layer consists of MongoDB and the Mongoose ODM. Mongoose schemas define the structure and validation rules for the users, files, permissions, and transfer_logs collections, and provide the methods the controllers use to query and update data.

File Storage and Logging

Encrypted file content is stored in a designated server-side storage directory (a cloud object store is discussed as a future enhancement). Logging occurs at two levels: HTTP-level request logging for debugging, and application-level transfer logs stored in MongoDB for accountability.

Communication Between Layers

The frontend and backend communicate via JSON over HTTP. Within the backend, modules communicate through function calls and shared middleware. The backend and MongoDB communicate through Mongoose. Because module boundaries are kept clean, replacing an in-process call with a network call in a future microservices version would require limited changes.

Table 4.1 – Layer Summary

Layer	Technology	Main Responsibility	Trust Level
Frontend	React.js	Presentation and input collection	Untrusted (runs on the user's device)
API / Backend	Node.js and Express.js	Security checks and business logic	Trusted
Encryption	Node.js crypto	Protect and recover file content	Trusted; holds key access
Database	MongoDB with Mongoose	Persistent metadata	Trusted; access restricted
File storage	Server storage directory	Hold encrypted files	Trusted; not web-accessible

The trust levels explain why validation and permission checks are repeated on the server even when the frontend already checks the same things. A user can bypass the frontend entirely, for example by sending requests from Postman, so only server-side checks count as security controls.

4.3 Deployment Architecture

For development and demonstration, the system is designed to run locally: the React development server, the Express.js server, and a MongoDB instance all run on the developer's machine. For hosting, the React application would be built into static files and served from a static hosting service, the Express.js server would run on a Node.js hosting environment, and MongoDB could run locally or on MongoDB Atlas. These are described as proposed deployment options; no real deployment is claimed in this document.

Figure 4.3 – Proposed Deployment Architecture (see also Chapter 9)

Detailed deployment steps, environment configuration, and security considerations are covered in Chapter 9.

Authentication, Upload and Download Workflows

Figure 9.1 — Deployment Architecture

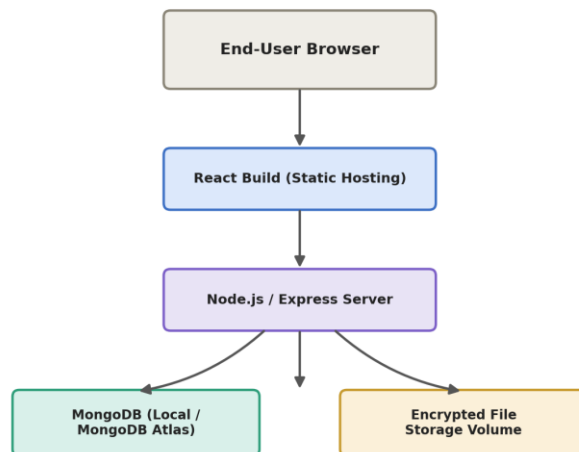
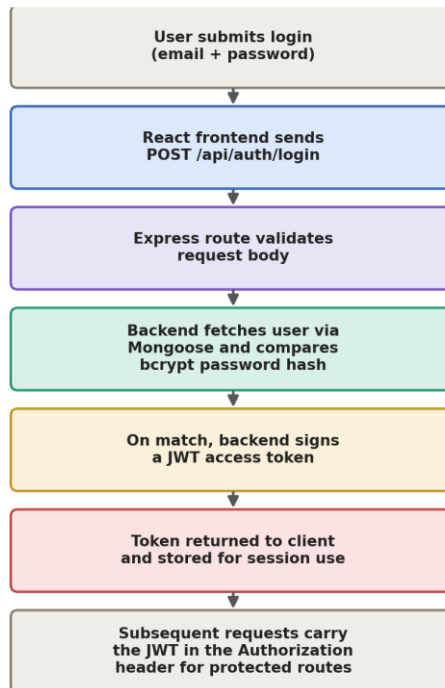


Figure 4.3 – Proposed Deployment Architecture (see also Chapter 9)

Detailed deployment steps, environment configuration, and security considerations are covered in Chapter 9.

Authentication, Upload and Download Workflows

Figure 4.3 — User Authentication Flow



.4 – User Authentication Flow

Figure 4.5 — File Download Workflow

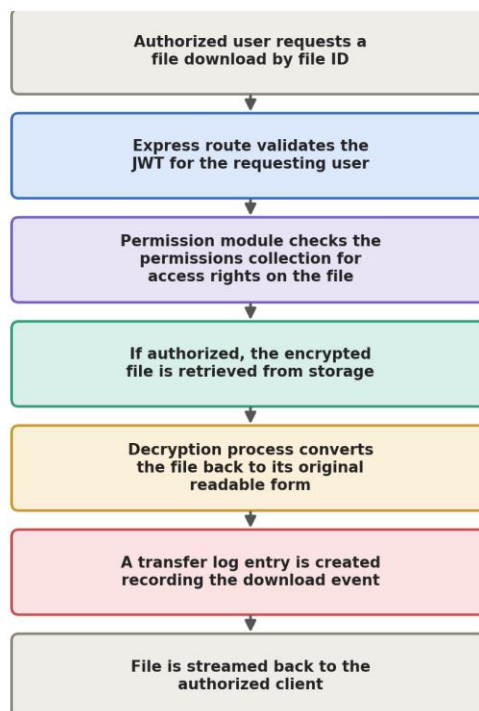


Figure 4.6 – File Download Workflow

Figures 4.4 to 4.6 summarise the three most important workflows. In each, the JWT check occurs first, permission or validation checks follow, and only then does the sensitive operation (encryption, decryption, or data retrieval) take place.

CHAPTER 5 – DESIGN

5.1 System Design

The system is designed as a set of cooperating modules, each with a single, clearly defined responsibility. This section describes the modules and the API surface that connects them.

Table 5.0 – Backend Modules and Responsibilities

Module	Responsibility
Authentication Module	Registration, login, password hashing, JWT issuing and verification
File Management Module	Upload handling, validation, metadata creation, file listing, download
Encryption Module	Encrypting files before storage and decrypting them for authorized users
Permission Module	Granting, revoking, and checking file access permissions
Database Module	Mongoose connection and schemas for MongoDB
Logging Module	Recording upload, download, and permission events in transfer_logs

Table 5.0a – Proposed REST API Endpoints

Method	Endpoint	Purpose	Auth
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Log in and receive a JWT	No
POST	/api/files/upload	Upload and encrypt a file	Yes
GET	/api/files	List files owned by or shared with the user	Yes
GET	/api/files/:id/download	Download and decrypt an authorized file	Yes
POST	/api/permissions	Grant a permission on a file	Yes (owner)
DELETE	/api/permissions/:id	Revoke a permission	Yes (owner)
GET	/api/logs	View transfer logs for the user's files	Yes

These endpoint names are a proposed design and may be adjusted during implementation. The design follows the principle of least privilege: each endpoint checks that the caller is allowed to perform the requested action, and owner-only actions (such as granting permissions) verify ownership explicitly.

Design Principles

- Least privilege: users can perform only the actions their role and permissions allow.
- Defence in depth: authentication, authorization, validation, and encryption are all applied, so the failure of one does not expose everything.
- Separation of concerns: each module has one responsibility.

- Statelessness: the server identifies the caller from the JWT on each request rather than remembering sessions.
- Fail safely: when a check fails or an error occurs, access is denied and a generic message is returned.

5.2 ER Diagram / Database Schema

MongoDB is a document database, so the design is expressed in terms of collections, documents, and fields rather than relational tables and rows. Figure 5.1 shows the four collections and the references between them.

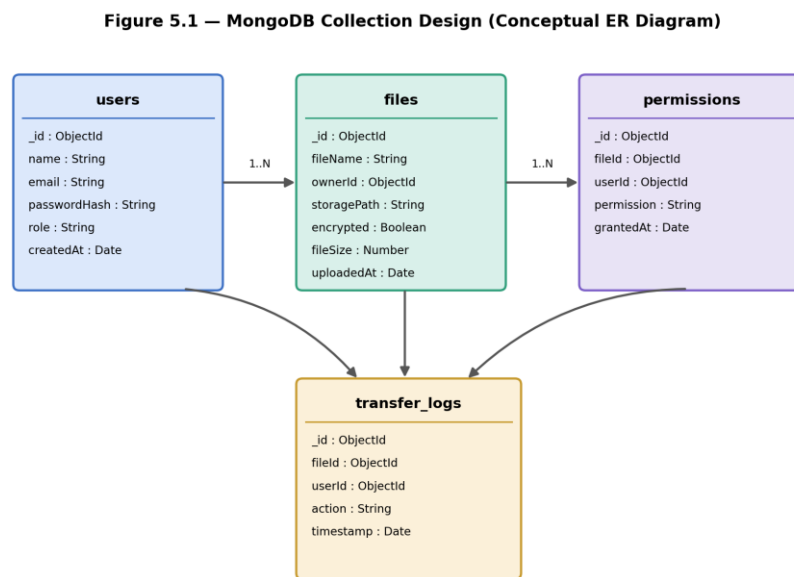


Figure 5.1 — MongoDB Collection Design

A user can own many files; a file can have many permission documents (one for each user granted access); and both files and users are referenced by transfer log documents. References are stored as ObjectId values, and Mongoose’s populate feature can be used to resolve them when needed. The choice of references over embedding keeps documents small and lets permissions and logs grow independently without inflating the file or user documents.

5.3 Database Collections

The following tables describe each collection. The example structures are conceptual schema illustrations and are not actual database records.

users

Purpose: stores registered accounts. The passwordHash field stores a bcrypt hash – never the plain password – and role supports authorization (for example, user or admin).

Table 5.1 – Users Collection Fields

Field	Type	Description
_id	ObjectId	Unique identifier generated by MongoDB
name	String	Full name of the user
email	String (unique)	Login identifier; indexed for fast lookup
passwordHash	String	bcrypt hash of the password
role	String	Authorization role, e.g. user or admin
createdAt	Date	Account creation time

```
{
  "_id": "...",
  "name": "...",
  "email": "...",
  "passwordHash": "...",
  "role": "user",
  "createdAt": "..."
}
```

files

Purpose: stores metadata about each uploaded file. The file content itself is not stored in this collection; storagePath points to the encrypted file in file storage.

Table 5.2 – Files Collection Fields

Field	Type	Description
_id	ObjectId	Unique file identifier
fileName	String	Original name shown to users
ownerId	ObjectId (ref users)	User who uploaded the file
storagePath	String	Location of the encrypted file in storage
encrypted	Boolean	Indicates the stored file is encrypted
fileSize	Number	Size of the file in bytes
uploadedAt	Date	Upload time

```
{
  "_id": "...",
  "fileName": "...",
  "ownerId": "...",
  "storagePath": "...",
  "encrypted": true,
  "fileSize": "...",
  "uploadedAt": "..."
}
```

permissions

Purpose: records which user may access which file and with what level of permission (for example, read). Download requests consult this collection.

Table 5.3 – Permissions Collection Fields

Field	Type	Description
_id	ObjectId	Unique permission identifier
fileId	ObjectId (ref files)	File the permission applies to
userId	ObjectId (ref users)	User receiving the permission
permission	String	Permission level, e.g. read

```
{
  "_id": "...",
  "fileId": "...",
  "userId": "...",
  "permission": "read"
}
```

transfer_logs

Purpose: an activity trail of uploads, downloads, and permission changes, useful for accountability and troubleshooting.

Table 5.4 – Transfer Logs Fields

Field	Type	Description
_id	ObjectId	Unique log identifier
fileId	ObjectId (ref files)	File involved in the event
userId	ObjectId (ref users)	User who performed the action
action	String	upload, download, share, revoke, etc.
timestamp	Date	Time of the event

```
{
  "_id": "...",
  "fileId": "...",
  "userId": "...",
  "action": "download",
  "timestamp": "..."
}
```

5.4 Data Flow / Workflow Design

The data flow of the system is organised around three primary workflows, shown in Chapter 4: authentication, upload, and download. This section describes the reasoning behind their design.

Authentication first: no file operation is available without a valid JWT, so the JWT check is a middleware step common to all protected routes. Upload next: because encryption must occur before the file is stored, the server processes the file in memory or a temporary location, encrypts it, and only then writes it to permanent storage; metadata is created only after storage succeeds so that MongoDB never references a file that does not exist. Download last: the permission check happens before the encrypted file is even read from storage, and the log entry is written when the download is served.

Figure 5.2 — Permission Management Flow

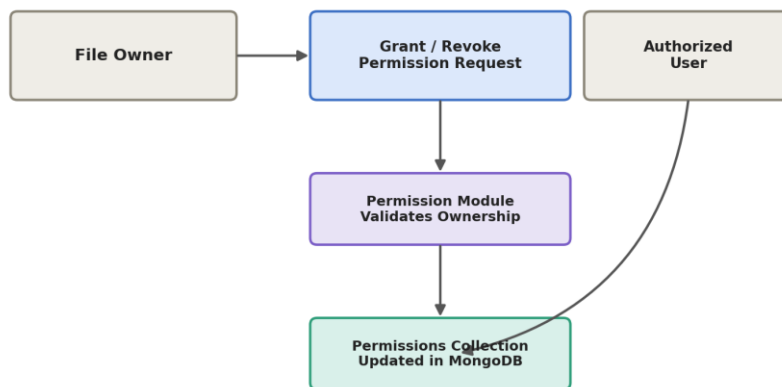


Figure 5.2 – Permission Management Flow

Figure 5.2 shows the permission-management flow: only the owner of a file may grant or revoke access, the Permission Module verifies ownership, and the permissions collection is updated accordingly. Every subsequent download by another user is checked against that collection.

5.5 User Interface Design

The user interface is designed to be simple and uncluttered, so that users can complete the primary tasks with few steps. The main screens are described below; actual screenshots are provided as placeholders in Appendix A.

Table 5.5 – Planned User Interface Screens

Screen	Purpose	Key Elements
Home Page	Introduce the system	Project title, login and register links
Registration	Create an account	Name, email, password, submit button, error messages
Login	Authenticate	Email, password, submit button, error messages
Dashboard	Overview of files	List of owned and shared files, upload button, logout
File Upload	Upload a confidential file	File chooser, upload button, progress and status message
Permission Management	Share with other users	User selector, permission level, grant/revoke buttons
Download	Retrieve an authorized file	Download button, error message if access is denied
Transfer Log	View activity	Table of actions, users, and timestamps

The layout is responsive so that it adapts to different screen sizes. Feedback messages (success,

validation error, unauthorized) appear near the relevant control, and protected screens redirect unauthenticated users to the login page.

5.6 Security Design

Security is treated as a cross-cutting concern rather than a single feature. The design combines several layers, because no single mechanism is sufficient on its own. It is also important to state clearly that encryption reduces the risk of unauthorized access to file contents when implemented correctly; it does not make a system “100% secure”.

Table 5.6 – Security Measures

Measure	Purpose	Where Applied
Authentication (JWT)	Confirm user identity on every protected request	Auth middleware
Password hashing (bcrypt)	Protect stored credentials; one-way	Registration and login
Authorization / access control	Restrict actions to owners and permitted users	Permission Module
File encryption (symmetric)	Protect file content at rest; reversible with the key	Encryption Module
Input validation	Reject malformed or malicious input	Route validators
File type and size restrictions	Limit unsafe or oversized uploads	Upload middleware
Environment variables	Keep secrets out of source code	Server configuration
HTTPS (deployment)	Protect data in transit	Hosting environment
Transfer logging	Traceability of file activity	Logging Module

Password Hashing Is Not File Encryption

Password hashing and file encryption are different tools for different purposes. bcrypt produces a one-way hash: the original password cannot be recovered from it, and login works by hashing the submitted password and comparing the results. File encryption, by contrast, is designed to be reversible: an authorized process holding the correct key can decrypt the file. bcrypt therefore does not encrypt files, and a JWT does not encrypt uploaded files; a JWT only proves identity and carries claims.

Key Handling

The encryption key is provided through an environment variable and is never committed to GitHub or stored in MongoDB. Loss or exposure of the key defeats the protection, so secure key management is identified as an important limitation and future enhancement in Chapters 10 and 11.

CHAPTER 6 – IMPLEMENTATION

This chapter describes how the system is proposed to be implemented. The descriptions represent the design approach and expected behaviour; the code snippets are representative examples rather than a claim about a finished codebase.

6.1 Module-Wise Implementation

The backend is structured as a conventional Express.js project. A representative folder layout is shown below.

```
server/  
  config/db.js           (Mongoose connection)  
  models/User.js, File.js, Permission.js, TransferLog.js  
  routes/authRoutes.js, fileRoutes.js, permissionRoutes.js  
  controllers/authController.js, fileController.js, permissionController.js  
  middleware/authMiddleware.js, uploadMiddleware.js, errorHandler.js  
  utils/encryption.js, logger.js  
  server.js  
client/  
  src/components/ (Register, Login, Dashboard, Upload, Permissions, Logs)  
  src/services/api.js  
  src/App.js
```

Each module owns its routes, controllers, and models. The Authentication Module (authRoutes and authController) covers registration and login. The File Module covers upload, listing, and download. The Permission Module covers granting, revoking, and checking permissions. The Encryption Module (utils/encryption.js) exposes two functions, encrypt and decrypt, used only by the File Module. The Logging Module exposes a function that creates a transfer_logs document.

This separation follows the single-responsibility principle and is the basis of the modular backend architecture described in Chapter 4.

Database Connection

```
// config/db.js  
const mongoose = require('mongoose');  
module.exports = async () => {  
  try {  
    await mongoose.connect(process.env.MONGODB_URI);  
    console.log('MongoDB connected');  
  } catch (err) {  
    console.error('MongoDB connection error:', err.message);  
    process.exit(1);  
  }  
};
```

The connection function is called once when the server starts. If the connection fails, the error is logged and the process stops so that the problem is noticed immediately rather than producing confusing errors later.

6.2 Front-End Logic

The React frontend uses functional components and hooks. Form components hold input values in state with `useState`; `useEffect` is used to load the file list when the dashboard mounts; and React Router defines routes for `/login`, `/register`, and `/dashboard`. A protected-route wrapper checks whether a token is present and redirects to the login page if it is not.

The JWT returned at login is kept in application state (with optional persistence in the browser's storage, a choice that involves a security trade-off discussed in Chapter 10) and attached to the Authorization header by an API helper. File upload uses the browser's `FormData` object so the file can be sent as `multipart/form-data`. Errors returned by the backend are displayed to the user rather than being hidden.

Figure 6.1 — Frontend-Backend Communication

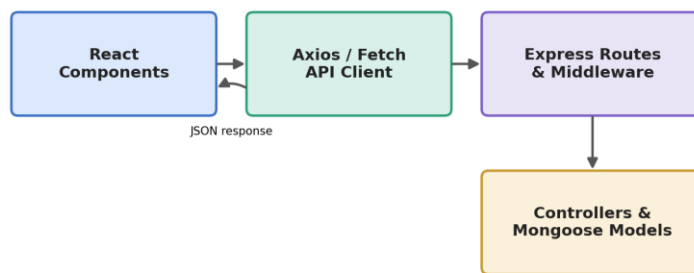


Figure 6.1 – Frontend–Backend Communication

Upload Component (simplified)

```
import { useState } from 'react';

export default function UploadForm({ onUploaded }) {
  const [file, setFile] = useState(null);
  const [message, setMessage] = useState('');

  const handleUpload = async (e) => {
    e.preventDefault();
    if (!file) return setMessage('Please choose a file');
    const data = new FormData();
    data.append('file', file);
    const res = await fetch(`${API_URL}/api/files/upload`, {
      method: 'POST',
      headers: authHeaders(),
      body: data,
    });
    setMessage(res.ok ? 'Upload successful' : 'Upload failed');
    if (res.ok) onUploaded();
  };

  return (
    <form onSubmit={handleUpload}>
      <input type="file" onChange={e => setFile(e.target.files[0])} />
    </form>
  );
}
```

```

        <button type="submit">Upload</button>
        <p>{message}</p>
    </form>
  );
}

```

The component keeps the chosen file and a status message in state, builds a FormData object, and sends it with the token. Note that the Content-Type header is not set manually, because the browser adds the correct multipart boundary automatically.

Protected Route

```

function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" replace />;
}

```

This wrapper improves the user experience by redirecting anonymous visitors, but real protection remains on the server because a token check in the browser can be bypassed.

6.3 API Integration

The frontend integrates with the backend through a small API service module that wraps each endpoint in a function, such as loginUser, registerUser, uploadFile, listFiles, downloadFile, grantPermission, and revokePermission. Centralising API calls means the base URL, headers, and error handling are defined once.

The backend enables Cross-Origin Resource Sharing (CORS) only for the frontend's origin so that the browser permits the frontend to call the API during development and deployment. Responses use consistent JSON shapes and appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 500) so the frontend can react correctly to each outcome. Postman is used to verify each endpoint before the corresponding frontend call is written.

API Service Module (Frontend)

```

// src/services/api.js
const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000';

function authHeaders() {
  const token = localStorage.getItem('token');
  return token ? { Authorization: `Bearer ${token}` } : {};
}

export async function listFiles() {
  const res = await fetch(`${API_URL}/api/files`, { headers: authHeaders() });
  if (!res.ok) throw new Error('Could not load files');
  return res.json();
}

```

Wrapping fetch calls in helpers means every request carries the token consistently. Storing a token in browser storage is convenient but exposes it to script injection attacks, so it is noted as a trade-off in Chapter 10.

6.4 Authentication and Authorization

User Registration

The flow is: React registration form, API request to POST /api/auth/register, Express route, validation of name, email, and password, hashing the password with bcrypt, saving the user through Mongoose to MongoDB, and returning a success response. Duplicate emails are rejected.

User Login

The flow is: React login form, POST /api/auth/login, lookup of the user by email, comparison of the submitted password with the stored hash using bcrypt, JWT generation on success, and return of the token to the frontend, which then treats the user as authenticated.

JWT Authentication

A JSON Web Token is a compact, signed token that carries claims such as the user's identifier and role. It is used so that the server can identify the caller on each request without storing session state. The server signs the token with a secret (JWT_SECRET) and sets an expiry time. A middleware reads the Authorization header, verifies the signature and expiry, and attaches the user identity to the request; invalid or missing tokens receive a 401 response. Protected routes are those that pass through this middleware.

Authorization

Authorization decides what an authenticated user may do. In this system it is based on roles (user or admin), file ownership, and per-file permissions. The owner of a file may always access it and manage its permissions; another user may download a file only if a matching permission document exists. Unauthorized requests receive a 403 response.

Registration Controller

```
exports.register = async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password || password.length < 8)
    return res.status(400).json({ message: 'Invalid input' });
  if (await User.findOne({ email }))
    return res.status(409).json({ message: 'Email already registered' });
  const passwordHash = await bcrypt.hash(password, 10);
  await User.create({ name, email, passwordHash });
  res.status(201).json({ message: 'Registered' });
};
```

The controller validates input, rejects duplicates, hashes the password with bcrypt, and stores only the hash. The number 10 is the cost factor, which controls how much work hashing requires and therefore how hard brute-force guessing is.

Permission Check Function

```
async function canAccess(fileId, userId) {
  const file = await File.findById(fileId);
  if (!file) return { ok: false, status: 404 };
  if (String(file.ownerId) === String(userId)) return { ok: true, file };
  const perm = await Permission.findOne({ fileId, userId });
  return perm ? { ok: true, file } : { ok: false, status: 403 };
}
```

6.5 File Upload and File Handling

File selection occurs in the React upload component. The frontend sends the file as multipart/form-data with the JWT. On the backend, an upload middleware (such as Multer) receives the file, checks that the file type is in an allowed list and that its size is under a configured limit, and rejects other files with a 400 response.

The validated file is passed to the Encryption Module, the encrypted output is written to server-side storage under a generated, non-guessable file name, a files document is created in MongoDB with the owner, size, and storage path, and a transfer log entry with action upload is recorded. The client receives the new file's ID. Only metadata is stored in MongoDB; file content is kept in file storage, which is an architectural choice rather than a claim that MongoDB never stores binary data.

Upload Controller

```
exports.uploadFile = async (req, res) => {
  if (!req.file) return res.status(400).json({ message: 'No file' });
  const allowed = ['application/pdf', 'image/png', 'image/jpeg', 'text/plain'];
  if (!allowed.includes(req.file.mimetype))
    return res.status(400).json({ message: 'Unsupported file type' });
  const encrypted = encrypt(req.file.buffer);
  const name = crypto.randomUUID() + '.enc';
  await fs.promises.writeFile(path.join(STORAGE_DIR, name), encrypted);
  const file = await File.create({
    fileName: req.file.originalname, ownerId: req.user.id,
    storagePath: name, encrypted: true, fileSize: req.file.size });
  await logEvent(file._id, req.user.id, 'upload');
  res.status(201).json({ fileId: file._id });
};
```

The stored file name is a random identifier with no relation to the original name, so the storage path reveals nothing about the content. The allowed type list is an example and should be chosen by the team.

6.6 File Encryption and Decryption

The plain file is read as bytes and encrypted with a symmetric algorithm using a key derived from the ENCRYPTION_KEY environment variable and a fresh random initialization vector (IV) for each file. The IV (and authentication tag when an authenticated mode such as AES-GCM is used) is stored alongside the encrypted data so that decryption is possible. The encrypted

bytes are written to storage.

Figure 6.2 — File Encryption Workflow

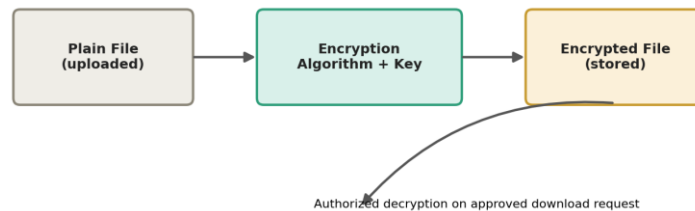


Figure 6.2 – File Encryption Workflow

For an authorized download, the backend reads the encrypted file, reads the IV and tag, and decrypts using the same key, streaming the recovered bytes to the client. No real key values appear in this document, and none should appear in source code. If the tag check fails, the file has been altered or the wrong key is being used, and the request fails with an error.

Download Controller

```
exports.downloadFile = async (req, res) => {
  const access = await canAccess(req.params.id, req.user.id);
  if (!access.ok) {
    await logEvent(req.params.id, req.user.id, 'denied');
    return res.status(access.status).json({ message: 'Access denied' });
  }
  const payload = await fs.promises.readFile(
    path.join(STORAGE_DIR, access.file.storagePath));
  const plain = decrypt(payload);
  await logEvent(access.file._id, req.user.id, 'download');
  res.setHeader('Content-Disposition',
    `attachment; filename="${access.file.fileName}"`);
  res.send(plain);
};
```

The order of operations matters: identity, then permission, then reading and decrypting, then logging and responding. Nothing sensitive is read until the permission check has passed.

6.7 MongoDB Integration

The flow for any database operation is: React component, API request, Express route, controller, Mongoose model, MongoDB. On startup the server connects to MongoDB using the MONGODB_URI environment variable. Mongoose schemas enforce field types, required fields, and uniqueness (for example, unique email), and provide methods such as create, findOne, find, and findByIdAndDelete. If the database is unreachable, the connection error is logged and requests that need the database return a 500 response.

Permission and Log Models

```
const permissionSchema = new mongoose.Schema({
  fileId: { type: mongoose.Schema.Types.ObjectId, ref: 'File', required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  permission: { type: String, enum: ['read'], default: 'read' },
});
permissionSchema.index({ fileId: 1, userId: 1 }, { unique: true });

const logSchema = new mongoose.Schema({
  fileId: { type: mongoose.Schema.Types.ObjectId, ref: 'File' },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  action: { type: String, required: true },
  timestamp: { type: Date, default: Date.now },
});
```

The compound unique index on `fileId` and `userId` prevents the same permission from being created twice and also makes permission lookups fast. Restricting the permission field to an enumerated list rejects unexpected values before they reach the database.

6.8 Access Control and Permission Management

Permission checks are implemented in a reusable function that, given a file ID and a user ID, returns true if the user owns the file or a permissions document exists for that pair. The download controller calls this function before reading the file. Owners grant access by sending the recipient's identifier and a permission level; the Permission Module verifies that the caller is the owner before creating or deleting a permissions document. Revocation takes effect immediately for subsequent requests because the check is performed on every download.

6.9 Transfer Logging

After each significant event – upload, download, permission grant, permission revocation, and denied access attempts – the controller calls the Logging Module, which inserts a document containing the file, user, action, and timestamp into `transfer_logs`. Owners can view the log entries for their files through the logs endpoint. The log is meant for accountability and troubleshooting; it does not itself prevent misuse.

```
// utils/logger.js
const TransferLog = require('../models/TransferLog');
exports.logEvent = (fileId, userId, action) =>
  TransferLog.create({ fileId, userId, action, timestamp: new Date() });
```

6.10 Error Handling

Table 6.1 – Error Scenarios and Handling

Scenario	HTTP Status	Handling
Invalid credentials	401	Generic message that does not reveal whether email or password was wrong
Invalid or missing token	401	Reject before reaching the controller

Scenario	HTTP Status	Handling
Unauthorized access to a file	403	Deny, log the attempt
Invalid file (type or size)	400	Reject upload with explanatory message
File not found	404	Return not-found response
Invalid API request	400	Validation error listing the problem
Database failure	500	Log error, return generic message
Unexpected server error	500	Central error handler returns safe response without stack trace

A central error-handling middleware ensures that internal details, such as stack traces and database messages, are not leaked to clients.

```
// middleware/errorHandler.js
module.exports = (err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ message: err.publicMessage || 'Server
error' });
};
```

CHAPTER 7 – FEATURES

7.1 List of Main Features

The system offers the features listed in Table 7.1. Each feature is described from the user's perspective and from a technical perspective in the sections that follow.

Table 7.1 – Main Features

No.	Feature	Summary
1	User registration	Create an account with hashed password storage
2	Login and authentication	Verify credentials and issue a JWT
3	Dashboard	View owned and shared files
4	File upload	Upload confidential files with validation
5	File encryption	Encrypt files before storage
6	File metadata	Store name, owner, size, and path in MongoDB
7	File listing	List accessible files
8	File sharing and permission management	Grant or revoke access for specific users
9	Authorized download with decryption	Return the readable file to permitted users
10	Transfer logs	Record uploads, downloads, and permission changes
11	Logout	End the session on the client
12	Error handling and unauthorized access prevention	Clear errors; deny unpermitted access

7.2 How Each Feature Works

User registration and login. The user enters details in a form; the backend validates them, hashes the password with bcrypt, and stores the account. At login the credentials are verified and a JWT is issued.

Dashboard and file listing. After login, the dashboard requests the list of files the user owns or has been granted access to. The backend queries the files and permissions collections and returns only accessible entries.

File upload and encryption. The user chooses a file and submits it. The backend validates the file, encrypts it, stores it, and records its metadata. The user sees a confirmation.

Sharing and permissions. The owner selects a file and a recipient and grants a permission level. The permissions collection is updated, and the recipient sees the file in their list.

Download and decryption. A permitted user requests a file. After the token and permission

checks, the backend decrypts the file and returns it. Each download is logged.

Transfer logs, logout, and error handling. Owners can review activity for their files. Logout clears the token on the client. Errors such as unauthorized access or invalid files produce clear messages.

Feature Details: User and Technical Perspectives

Table 7.3 – Feature Details

Feature	User Perspective	Technical Perspective
Registration	Fills a short form and gets an account	Input validated; password hashed by bcrypt; user document created via Mongoose
Login	Enters email and password	Hash compared; JWT signed with JWT_SECRET and expiry
Authentication	Stays signed in during a session	Token verified in middleware on protected routes
Dashboard	Sees own and shared files	Query of files by owner and by permissions
File upload	Chooses a file and submits	Multipart request; validation; storage under random name
File encryption	Nothing extra to do	Symmetric encryption with per-file IV; key from environment
File metadata	Sees name, size, date	files document with ownerId, storagePath, fileSize
File listing	Sees only accessible files	Filtered by ownership and permission documents
File sharing	Picks a user and permission	permissions document created by owner
Authorized download	Clicks download and receives the file	Permission check, decryption, streamed response
Transfer logs	Reviews activity	transfer_logs queried by file
Logout	Signs out	Client discards the token; token also expires on its own
Error handling	Sees clear messages	Consistent status codes and central handler
Unauthorized access prevention	Denied when not permitted	403 response and a denied log entry

7.3 User Perspective

From a user's point of view, the system requires only a few steps: register, log in, upload a file, optionally share it with a colleague, and download files shared by others. The user does not need to understand encryption; the protection happens automatically. If the user tries something not allowed, the interface explains what went wrong.

7.4 Technical Description

Table 7.2 – Feature to Technical Mapping

Feature	Frontend	Backend	Database
Registration	Registration form	Validate, bcrypt hash	users
Login	Login form, token handling	bcrypt compare, JWT sign	users
Upload	FormData upload	Multer, encryption, storage	files, transfer_logs
Permissions	Permission UI	Ownership check	permissions, transfer_logs
Download	Download action	JWT, permission check, decryption	files, permissions, transfer_logs
Logs	Log table	Query by owner	transfer_logs

CHAPTER 8 – TESTING

This chapter defines the testing strategy and test cases. Because the system is documented at the design and proposed-implementation stage, actual execution results are not stated; results are marked “To Be Verified During Execution” and must be filled in by the team after running the tests.

8.1 Postman Testing

Postman is used to test the REST APIs independently of the frontend. A Postman collection is organised into folders for Authentication, Files, Permissions, and Logs. For each request the team records the method, URL, headers (including Authorization: Bearer <token>), body, and expected status code.

- Register and login requests verify that a token is returned only for valid credentials.
- Upload requests use the form-data body type with a file field.
- Download requests are tested with an owner token, a permitted user token, and an unauthorized user token.
- Requests with missing, expired, or tampered tokens verify 401 handling.

[INSERT POSTMAN TEST SCREENSHOT HERE]

Table 8.2 – Planned Postman Requests

Request	Method and URL	Authorization	Expected Status
Register	POST /api/auth/register	None	201
Register duplicate	POST /api/auth/register	None	409
Login	POST /api/auth/login	None	200 with token
Login wrong password	POST /api/auth/login	None	401
Upload file	POST /api/files/upload	Bearer token	201
List files	GET /api/files	Bearer token	200
Grant permission	POST /api/permissions	Owner token	201
Download (permitted)	GET /api/files/:id/download	Permitted token	200
Download (not permitted)	GET /api/files/:id/download	Other token	403
Download without token	GET /api/files/:id/download	None	401

A sample login request body that could be used in Postman is shown below. The values are placeholders and are not real credentials.

```
{  
  "email": "<test_user_email>",  
  "password": "<test_password>"  
}
```

8.2 Frontend/Backend Testing

Frontend testing checks that forms validate input, that error and success messages are displayed, that protected routes redirect unauthenticated users, and that the interface remains usable at different screen sizes. Backend testing checks each controller and middleware function: password hashing and comparison, token signing and verification, file validation, encryption and decryption round-trips, and permission checks. Optional automated tests may use a framework such as Jest, though the team's primary approach is manual and Postman-based.

Table 8.3 – Frontend and Backend Checks

Area		Method
Frontend		Manual
Frontend		Manual
Frontend		Manual, browser developer tools
Backend		Postman and Compass
Backend		Script or unit test
Backend		Postman
Backend		Postman

8.3 Integration Testing

Integration testing verifies that the pieces work together: React to Express to Mongoose to MongoDB. Key end-to-end scenarios include register then login then upload then share then download by another user, and unauthorized download attempts. The team confirms that data written by the backend appears correctly in MongoDB Compass and that the downloaded file matches the original.

8.4 Database Testing

Database testing confirms that Mongoose schemas enforce required fields and unique emails, that references (ownerId, fileId, userId) are stored correctly, that indexes exist, and that documents in each collection match the designed structure. Failure behaviour is tested by stopping MongoDB and checking that the server logs the error and returns a 500 response without crashing.

[INSERT MONGODB COMPASS SCREENSHOT HERE]

8.5 Security Testing

Security testing checks that stored passwords are hashes, that encrypted files in storage are not readable as the original content, that invalid or expired tokens are rejected, that users cannot download files without permission, that oversized and disallowed files are rejected, and that error responses do not leak internal details. These checks are basic functional security tests and do not constitute a professional penetration test or a security certification.

Table 8.4 – Security Checks

Threat	Check	Expected Outcome
Credential theft from database	Inspect users collection	Only hashes are stored
Direct storage access	Open a stored file in a viewer	Content is unreadable
Forged or expired token	Send altered token	401 response
Access to another user's file	Request download without permission	403 response and denied log entry
Malicious upload	Upload disallowed or oversized file	Rejected
Information leakage	Trigger an error	No stack trace or database detail in response

8.6 Test Cases and Results

Table 8.1 lists the planned test cases. The Actual Result column is intentionally left as “To Be Verified During Execution”; no results have been fabricated.

Table 8.1 – Test Cases

ID	Scenario	Input	Expected Result	Actual Result	Status
TC01	Successful user registration	Valid name, email, password	User created; password stored as hash; success response	To Be Verified During Execution	Pending
TC02	Registration with existing email	Email already registered	Request rejected with duplicate-email error	To Be Verified During Execution	Pending
TC03	Successful login	Valid email and password	JWT returned; user reaches dashboard	To Be Verified During Execution	Pending
TC04	Invalid login	Correct email, wrong password	401 response; no token issued	To Be Verified During Execution	Pending
TC05	File upload	Allowed file type within size limit	File encrypted and stored; metadata saved; success response	To Be Verified During Execution	Pending
TC06	Unsupported file type	File of a disallowed type	Upload rejected with 400	To Be Verified During Execution	Pending

ID	Scenario	Input	Expected Result	Actual Result	Status
TC07	File size exceeds limit	File larger than configured limit	Upload rejected with error	To Be Verified During Execution	Pending
TC08	File encryption	Uploaded file inspected in storage	Stored content is not readable as the original	To Be Verified During Execution	Pending
TC09	Unauthorized file access	User without permission requests download	403 response; event logged	To Be Verified During Execution	Pending
TC10	Authorized file download	Owner or permitted user requests download	File decrypted and returned identical to original	To Be Verified During Execution	Pending
TC11	Invalid JWT	Tampered or expired token	401 response	To Be Verified During Execution	Pending
TC12	File not found	Non-existent file ID	404 response	To Be Verified During Execution	Pending
TC13	MongoDB connection failure	Database stopped	Error logged; 500 response; no crash	To Be Verified During Execution	Pending
TC14	Invalid API request	Missing required fields	400 validation error	To Be Verified During Execution	Pending
TC15	Transfer log creation	Upload then download a file	Log entries for both actions exist	To Be Verified During Execution	Pending
TC16	Logout	Click logout	Token cleared; protected pages redirect to login	To Be Verified During Execution	Pending

After execution, the team should replace the Actual Result entries with observed outcomes and set Status to Pass or Fail. Any failed case should be traced to its cause, fixed, and re-run, and the outcome recorded. Attaching screenshots of Postman responses in Appendix D provides supporting evidence.

CHAPTER 9 – DEPLOYMENT

9.1 Steps to Deploy

The following steps describe local deployment and the proposed path to hosting. No live deployment is claimed.

- Install Node.js (LTS), npm, Git, and MongoDB (or create a MongoDB Atlas cluster).
- Clone the repository from GitHub.
- In the server folder, run npm install and create a .env file with the required variables.
- Start MongoDB and run the backend (for example, npm start or npm run dev).
- In the client folder, run npm install and then npm run dev (or npm start) to launch the React development server.
- For hosting, run the frontend build command to generate static files, and deploy the backend on a Node.js host with the environment variables configured.

```
git clone <your_repository_url>

# Backend
cd server
npm install
npm run dev

# Frontend (in another terminal)
cd client
npm install
npm run dev    # or npm start, depending on the project setup

# Production build of the frontend
npm run build
```

The commands above are typical for a Node.js project; the exact script names depend on how the team configures package.json and the React tooling.

9.2 Environment Configuration

Table 9.1 – Environment Variables

Variable	Placeholder	Purpose
PORT	<port_number>	Port on which Express listens
MONGODB_URI	<your_mongodb_connection_string>	Database connection
JWT_SECRET	<your_secret>	Signing key for JWTs
ENCRYPTION_KEY	<your_encryption_key>	Key used by the Encryption Module

9.3 Frontend Hosting

The React application is compiled with `npm run build` into static HTML, CSS, and JavaScript that any static hosting service can serve. Node.js and npm are needed at build time but not to serve the static files. The API base URL is configured for the production backend.

Static hosting is well suited to a React build because the output is only files. The production build reads the API address from an environment variable set at build time, so the same source can target a local backend during development and a hosted backend later. After deployment, the team should confirm that page refreshes on client-side routes work, which usually requires the host to redirect unknown paths to `index.html`.

9.4 Backend Hosting

The Express.js server runs on any environment that supports Node.js. Environment variables are set through the host's configuration rather than a committed file, CORS is restricted to the deployed frontend origin, and HTTPS is enabled so that data in transit is protected. Encrypted files require a persistent storage volume; ephemeral file systems would lose them.

Before exposing the backend, the team should confirm that the storage directory is outside any publicly served folder, that request body size limits are set, and that logs do not contain secrets. A process manager or the host's own supervision should restart the server if it stops. Health-check endpoints are a common addition, although they are not part of the current design.

9.5 MongoDB Deployment

During development MongoDB runs locally and is inspected with MongoDB Compass. MongoDB Atlas is an optional cloud alternative; using it involves creating a cluster, a database user, and a network-access rule, and placing the resulting connection string in `MONGODB_URI`. Access should be limited to the backend, and credentials must not be shared publicly.

Table 9.2 – Local versus Atlas MongoDB

Aspect	Local MongoDB	MongoDB Atlas (optional)
Setup	Install on the developer machine	Create a hosted cluster
Access	Localhost only	Network access rules and database user needed
Inspection	MongoDB Compass	MongoDB Compass or Atlas web interface
Suitability	Development and demonstration	Shared or hosted use

9.6 Environment Variables and Security

Secrets such as `JWT_SECRET`, `ENCRYPTION_KEY`, and the database URI must never be committed to GitHub. A `.gitignore` entry for the `.env` file prevents this, and a `.env.example` file containing only placeholders documents what is required. If a secret is ever exposed, it should be rotated immediately; note that rotating `ENCRYPTION_KEY` requires re-encrypting existing files.

```
# .env.example (placeholders only)
PORT=<port_number>
MONGODB_URI=<your_mongodb_connection_string>
JWT_SECRET=<your_secret>
ENCRYPTION_KEY=<your_encryption_key>

# .gitignore
node_modules/
.env
```

CHAPTER 10 – CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The following are realistic challenges anticipated for a project of this type. They are described as expected challenges and do not report specific incidents.

Table 10.1 – Challenges and Solutions

Challenge	Cause	Solution / Approach	Remaining Limitation
File encryption and decryption	Correct handling of keys, IVs, and binary data	Use Node.js crypto with a fresh IV per file; round-trip tests	Not independently audited
Secure key management	Key must be available to the server but hidden from others	Environment variable, not in code or database	No key rotation or hardware protection
Large file handling	Memory use when files are read whole	Enforce size limits; consider streaming	Very large files unsupported
File validation	Extensions can be spoofed	Check type and size at upload	No malware scanning
MongoDB schema design	Choosing references versus embedding	Separate collections with ObjectId references	Extra queries for joins
Authentication and authorization	Consistent checks on every route	Central middleware and permission function	Basic roles only
React and backend communication	Token handling and request formats	API service module; Postman verification first	Token storage trade-offs
CORS errors	Different frontend and backend origins	Configure CORS for the frontend origin	Must be updated per environment
Environment variables	Missing or wrongly set values	.env.example and startup checks	Manual configuration
Database connectivity and network failures	Service down or unreachable	Error handling and clear responses	No automatic failover
Secure file storage	Server-side storage can be lost or exposed	Encrypted content, restricted directory	Single-server storage

Detailed Discussion of Selected Challenges

Encryption and decryption. Problem: a downloaded file may be corrupt or fail to open. Cause: using a different key or initialization vector on decryption, or handling text encoding on binary data. Solution: treat all file data as Buffers, store the IV and authentication tag with the ciphertext, and test round trips. Remaining limitation: the implementation has not been reviewed by a cryptography specialist.

Secure key management. Problem: the server needs the key to work, yet the key must not be exposed. Cause: any secret usable by software can be exposed if the server is compromised.

Solution: keep the key in an environment variable outside version control. Remaining limitation: a single key protects all files, and rotation requires re-encrypting stored files.

Authorization and permissions. Problem: some routes may forget to check permissions. Cause: repeating checks by hand in each controller. Solution: use one shared function and test it with several users. Remaining limitation: only a read permission level is designed.

Frontend and backend communication. Problem: requests fail in the browser but succeed in Postman. Cause: CORS rules and missing Authorization headers. Solution: configure CORS and centralise header creation. Remaining limitation: origins must be updated for each environment.

Database connectivity. Problem: the server starts but queries hang or fail. Cause: MongoDB not running or a wrong connection string. Solution: log connection errors clearly and stop on failure. Remaining limitation: there is no automatic retry or failover.

10.2 Solutions Applied

The approaches above share common themes: keep security decisions on the server; validate all input; keep secrets out of source code; test each API in Postman before connecting the frontend; and keep modules small and focused so that problems can be isolated. Using Git branches per feature also reduced integration conflicts among team members.

Approach to Selected Challenges

Encryption correctness: the team tests a round trip – encrypt a file, decrypt it, and compare it byte for byte with the original – and also confirms that changing the wrong key or altering the stored bytes causes decryption to fail. Key management: the key is generated once using a cryptographically secure random source, stored outside the repository, and documented as a placeholder only. Large files: an upload size limit protects the server, and streaming is noted as an optimisation.

Permission logic: a single `canAccess` function is used by every route that reads a file, so the rule is written once and tested once. CORS: the backend permits only the known frontend origin. Environment variables: the server checks at startup that required variables exist and stops with a clear message if not.

10.3 Current Limitations / Known Issues

- The backend is modular but runs as a single service; it is not a production microservices system.
- Encryption uses one server-held key; there is no per-user key or key rotation.

- There is no multi-factor authentication, email verification, or password reset.
- Files are stored on a single server rather than in redundant cloud storage.
- There is no malware scanning of uploaded files.
- Testing results have not yet been recorded; the test cases must be executed and results filled in.
- The system has not been independently security-audited, and no security certification is claimed.

These limitations are consistent with the aims of an academic project and are listed openly so that readers can judge the work accurately. Several of them, especially key management and single-server storage, are addressed by the enhancements described in Chapter

CHAPTER 11 – FUTURE ENHANCEMENTS

11.1 Planned Features

The following are future enhancements only; none is claimed to be implemented.

Table 11.1 – Future Enhancements

Enhancement	Description
Multi-factor authentication	Second verification step at login
Email verification and password reset	Confirm ownership of email and allow recovery
One-time download links	Links that work once
File expiration	Automatic removal or access expiry after a set time
Advanced role-based access control	Finer-grained roles and groups
Virus/malware scanning	Scan uploads before storing
Chunked file upload	Reliable upload of large files
File versioning	Keep previous versions of a file
Advanced audit logs and monitoring	Richer logs, dashboards, alerts

Multi-factor authentication would add a second factor such as a one-time code, reducing the risk from stolen passwords. Email verification and password reset would confirm that an email address belongs to the user and provide a safe way to recover an account. One-time download links and file expiry would limit how long a file remains reachable. Advanced role-based access control would allow groups and finer roles than user and admin.

Malware scanning would inspect uploads before they are stored. Chunked upload would split large files into pieces so that uploads can resume after interruption. File versioning would keep earlier versions and support rollback. Advanced audit logs and monitoring would add dashboards and alerts on unusual activity.

11.2 Possible Integrations

Possible integrations include cloud object storage for encrypted files, real-time notifications when a file is shared, a mobile application using the same REST API, and a dedicated key-management service for storing encryption keys more securely.

Table 11.3 – Possible Integrations

Integration	Benefit
Cloud object storage	Durable, scalable storage for encrypted files
Key-management service	Safer storage and rotation of encryption keys

Integration	Benefit
Email service	Verification, password reset, and share notifications
WebSocket notifications	Real-time alerts when a file is shared
Mobile application	Access through the same REST API

11.3 Possible Optimizations

Optimizations include streaming encryption and decryption to reduce memory use, caching permission lookups, adding database indexes based on observed queries, and paginating file lists and logs.

Streaming encryption would process a file in pieces so that memory use stays low even for larger files. Caching recent permission results could reduce database queries, provided that revocation is handled correctly so that stale cache entries never allow access. Pagination would keep the file list and log views fast as records grow, and query profiling in MongoDB Compass would guide which indexes to add.

11.4 Scalability Improvements

To scale, the stateless JWT design allows several backend instances to run behind a load balancer (horizontal scaling). The logical modules could be separated into independently deployed microservices, and MongoDB could be deployed as a replica set or sharded cluster. These changes would turn the modular design into a genuinely distributed deployment.

Table 11.2 – Path from Modular to Distributed Deployment

Stage	Description
Current design	Logical modules in one Node.js service
Step 1	Run several identical instances behind a load balancer (horizontal scaling)
Step 2	Move encrypted files to cloud object storage
Step 3	Extract authentication and file services as separate deployable services
Step 4	Use a MongoDB replica set or sharded cluster and central monitoring

All stages in this table are future possibilities and are not part of the current design.

CHAPTER 12 – CONCLUSION

12.1 Summary of the Project

Traditional file-transfer methods can expose confidential files to unauthorized users during storage or transmission. This project proposes a solution: a secure, database-driven file transfer system with a React.js frontend, a Node.js and Express.js backend, and a MongoDB database accessed through Mongoose. Users authenticate with JWTs, passwords are protected with bcrypt, files are encrypted before storage, access is controlled through owner-managed permissions, and file activity is recorded in transfer logs.

The backend is organised into logical modules – authentication, file management, encryption, permission, database, and logging – illustrating distributed backend concepts and providing a path to independently deployed services in future. MongoDB stores users, file metadata, permissions, and logs, while encrypted file content is held in file storage.

The project also strengthened understanding of a key security distinction. Passwords are protected with a one-way hash, while files are protected with reversible encryption so that authorized users can retrieve them. Combining these with token-based authentication and owner-managed permissions gives a layered defence in which each mechanism covers a different risk.

12.2 What Was Achieved

The project produced a complete design of the system: requirements, technology choices, architecture with workflow diagrams, a four-collection MongoDB schema, security design, an implementation plan for each module, a testing strategy with sixteen test cases, deployment guidance, and an analysis of challenges, limitations, and future work. Encryption reduces the risk of unauthorized access to file contents when implemented correctly, and it works together with authentication, authorization, secure key handling, validation, and secure APIs rather than replacing them.

Table 12.1 – Objectives and Their Treatment

Objective	How It Was Addressed in the Design
Secure user authentication	JWT authentication with bcrypt password hashing
Authorization	Roles, ownership, and per-file permissions
File encryption and decryption	Symmetric encryption module with key from environment
MongoDB database management	Four collections defined with Mongoose schemas
REST API development	Endpoints for auth, files, permissions, and logs
React frontend	Component-based interface with protected routes

Objective	How It Was Addressed in the Design
Transfer logging	Log documents for upload, download, and permission events
Modular backend concept	Six logical modules with a path to microservices

It should be emphasised that this document presents the design and proposed implementation. Statements about behaviour describe the intended system, and test outcomes must be recorded by the team when the tests are executed.

12.3 Skills Learned During Development

- Designing MongoDB collections and references with Mongoose.
- Building REST APIs with Node.js and Express.js.
- Implementing JWT authentication and bcrypt password hashing.
- Understanding the difference between hashing and encryption.
- Building React components, forms, and routing.
- Testing APIs with Postman and planning test cases.
- Collaborating with Git and GitHub.
- Documenting a software project in academic format.

Beyond technical skills, the team practised dividing work among members, reviewing each other's changes, preparing for project reviews, and writing structured technical documentation. The future scope of the work includes stronger key management, multi-factor authentication, cloud-based storage, improved auditing, file expiry, and deployment as independently scalable services.

CHAPTER 13 – REFERENCES

Official Documentation

- [1] React Documentation – <https://react.dev>
- [2] Node.js Documentation – <https://nodejs.org/docs>
- [3] Express.js Documentation – <https://expressjs.com>
- [4] MongoDB Documentation – <https://www.mongodb.com/docs>
- [5] Mongoose Documentation – <https://mongoosejs.com/docs>
- [6] Postman Learning Center – <https://learning.postman.com>
- [7] Git Documentation – <https://git-scm.com/doc>
- [8] GitHub Docs – <https://docs.github.com>

Web and Security References

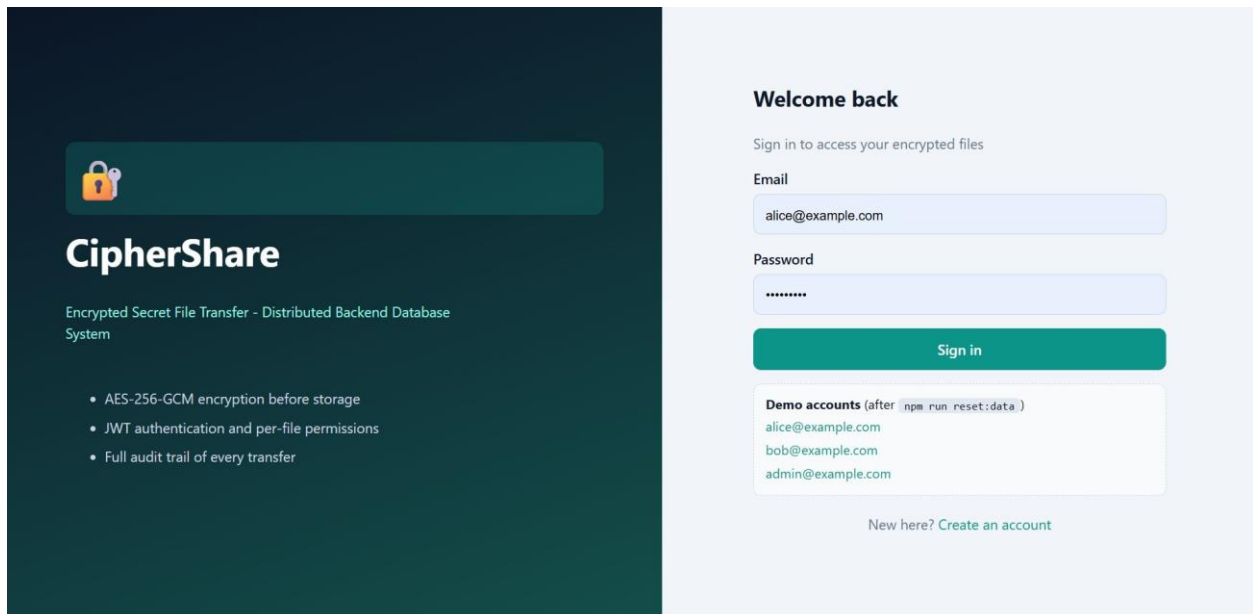
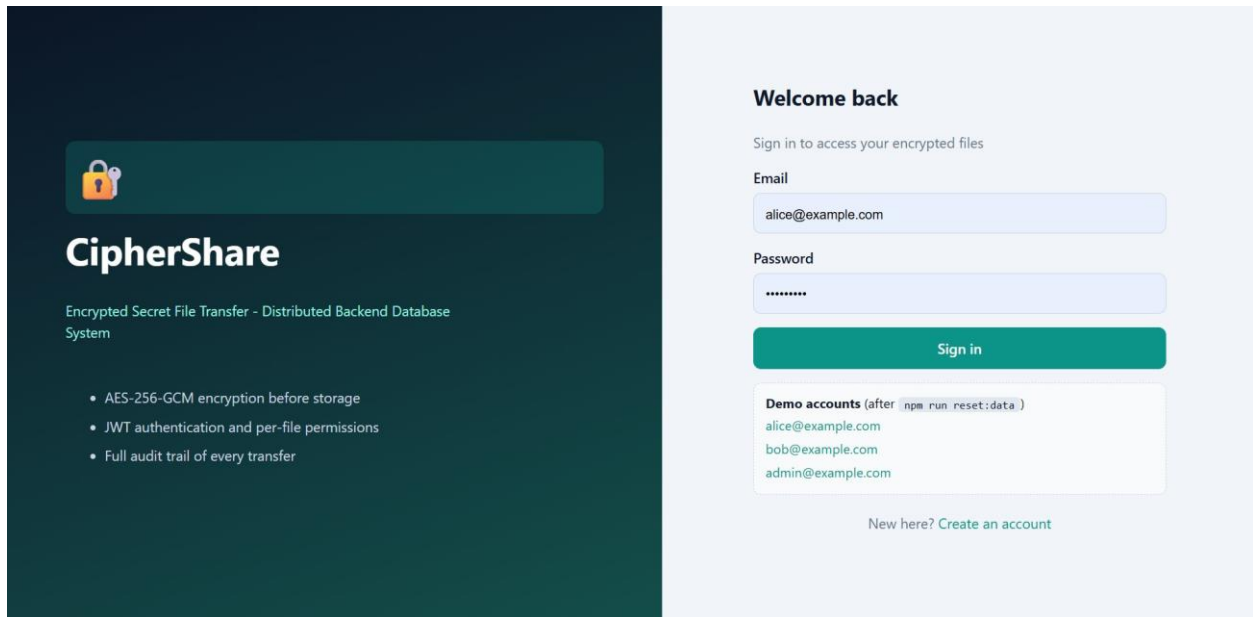
- [1] MDN Web Docs – <https://developer.mozilla.org>
- [2] OWASP Foundation – <https://owasp.org>
- [3] JSON Web Token (JWT), RFC 7519 – <https://www.rfc-editor.org/rfc/rfc7519>
- [4] Node.js crypto module documentation – <https://nodejs.org/api/crypto.html>
- [5] bcrypt package on npm – <https://www.npmjs.com/package/bcrypt>
- [6] Multer package on npm – <https://www.npmjs.com/package/multer>

Only official documentation and standards have been listed. Team members should verify the URLs and add any books, tutorials, or course materials act

CHAPTER 14 – APPENDICES

APPENDIX A – APPLICATION SCREENSHOTS

Screenshots are to be inserted after the application is implemented. No fake screenshots are included.



 **CipherShare**
Encrypted file transfer

Dashboard

My Files

Shared With Me

Upload

Activity Logs

Profile

System Status

Logout

A

Alice Johnson
User

Encrypted Secret File Transfer

+ Upload

My Files

Search files...

+ Upload

Files you uploaded - stored encrypted

FILE NAME	OWNER	SIZE	TYPE	DATE	ACCESS	SHARED WITH	ACTIONS
 secret.txt	Alice Johnson	21 B	TXT	Sep 30, 2026, 10:44 PM	Owner	Private	DetailsDownloadShareDelete
 project-notes.md	Alice Johnson	63 B	MD	Sep 30, 2026, 10:37 PM	Owner	Private	DetailsDownloadShareDelete
 demo-secret.txt	Alice Johnson	88 B	TXT	Sep 30, 2026, 10:37 PM	Owner	bob@example.com	DetailsDownloadShareDelete

 **CipherShare**
Encrypted file transfer

Dashboard

My Files

Shared With Me

Upload

Activity Logs

Profile

System Status

Logout

A

Alice Johnson
User

Encrypted Secret File Transfer

+ Upload

Shared With Me

Search files...

Files other users granted you access to



Nothing shared with you yet

When someone shares a file with your email, it appears here.

 **CipherShare**
Encrypted file transfer

Dashboard

My Files

Shared With Me

Upload

Activity Logs

Profile

System Status

Logout

A

Alice Johnson
User

Encrypted Secret File Transfer

+ Upload

Upload a secret file

The server encrypts it with AES-256-GCM before anything is written to disk



Drag & drop a file here, or click to browse
Max 10 MB - allowed: txt, md, csv, json, pdf, png, jpg, gif, doc(x), xls(x), ppt(x), zip

Clear

Encrypt & upload

What happens next? A fresh random key encrypts your file; that key is itself encrypted with the server master key. Only the ciphertext is stored on disk, and every upload is written to the audit log.

 **CipherShare**
Encrypted file transfer

Dashboard

My Files

Shared With Me

Upload

Activity Logs

Profile

System Status

Logout

A

Alice Johnson
User

Encrypted Secret File Transfer

+ Upload

Activity & transfer logs

All actions

Refresh

Uploads, downloads, sharing and revocations involving you or your files

USER	FILE	ACTION	TARGET	DATE/TIME	STATUS
Alice Johnson	-	Login	-	Oct 2, 2026, 6:33 PM	success
Alice Johnson	secret.txt	Upload	-	Sep 30, 2026, 10:44 PM	success
Alice Johnson	-	Login	-	Sep 30, 2026, 10:42 PM	success
Bob Smith	demo-secret.txt	Download	-	Sep 30, 2026, 10:07 PM	success
Alice Johnson	demo-secret.txt	Shared	bob@example.com	Sep 30, 2026, 10:02 PM	success
Alice Johnson	demo-secret.txt	Upload	-	Sep 30, 2026, 9:57 PM	success
Alice Johnson	-	Registered	-	Sep 30, 2026, 9:39 PM	success

Showing 7 of 7 events.

**CipherShare**
Encrypted file transfer

Dashboard

My Files

Shared With Me

Upload

Activity Logs

Profile

System Status

Logout

**Alice Johnson**
User

Encrypted Secret File Transfer

+ Upload

Profile

Your account and security details

**Alice Johnson**
alice@example.com

Role

user

Member since

Sep 30, 2026, 10:37 PM

Last login

Oct 2, 2026, 6:33 PM

Log out

How your account is protected

- Password stored only as a **bcrypt** hash
- Sessions use short-lived **JWT** tokens
- Files encrypted with **AES-256-GCM**
- Every action recorded in the audit log

APPENDIX B – SAMPLE CODE SNIPPETS

The following snippets are representative and illustrative; they should be adapted to the actual codebase.

B.1 React Login Component (simplified)

```
import { useState } from 'react';
import { loginUser } from '../services/api';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const data = await loginUser(email, password);
      localStorage.setItem('token', data.token);
    } catch (err) {
      setError('Invalid credentials');
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      <input type="password" value={password} onChange={(e) =>
setPassword(e.target.value)} />
      <button type="submit">Login</button>
      {error && <p>{error}</p>}
    </form>
  );
}
```

B.2 Login API Request

```
export async function loginUser(email, password) {
  const res = await fetch(`${API_URL}/api/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });
  if (!res.ok) throw new Error('Login failed');
  return res.json();
}
```

B.3 Express Route and Login Controller

```
// routes/authRoutes.js
router.post('/login', login);

// controllers/authController.js
exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash)))
    return res.status(401).json({ message: 'Invalid credentials' });
  const token = jwt.sign({ id: user._id, role: user.role },
    process.env.JWT_SECRET, { expiresIn: '1h' });
  res.json({ token });
};
```

B.4 Authentication Middleware

```
module.exports = (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ message: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) {
    res.status(401).json({ message: 'Invalid token' });
  }
};
```

B.5 Mongoose Model

```
const fileSchema = new mongoose.Schema({
  fileName: { type: String, required: true },
  ownerId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  storagePath: { type: String, required: true },
  encrypted: { type: Boolean, default: true },
  fileSize: Number,
  uploadedAt: { type: Date, default: Date.now },
});
module.exports = mongoose.model('File', fileSchema);
```

B.6 File Upload Route

```
const upload = multer({ storage: multer.memoryStorage(),
  limits: { fileSize: 10 * 1024 * 1024 } });
router.post('/upload', auth, upload.single('file'), uploadFile);
```

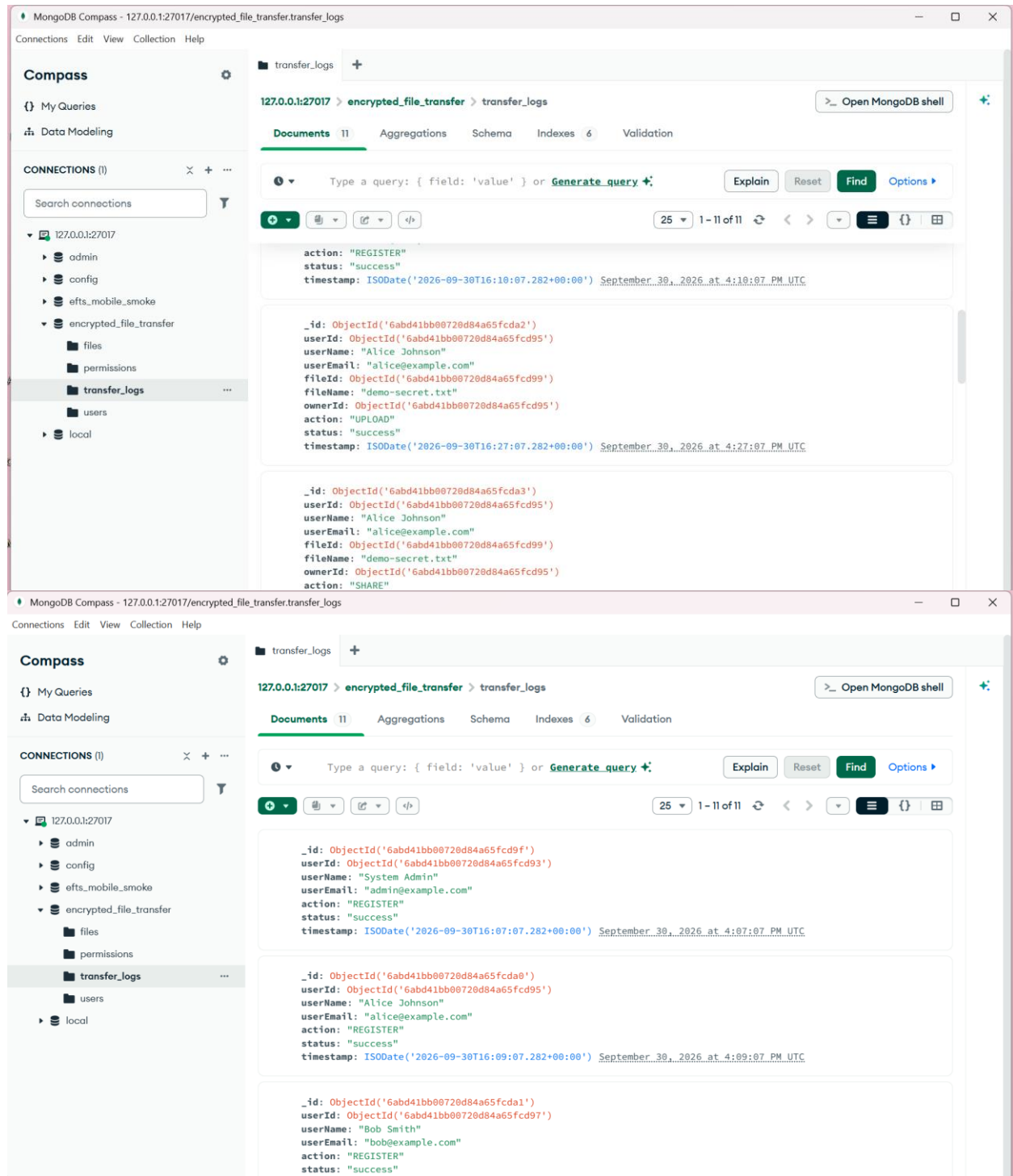
B.7 Encryption and Decryption (AES-256-GCM)

```
const crypto = require('crypto');
const key = Buffer.from(process.env.ENCRYPTION_KEY, 'hex'); // 32 bytes

exports.encrypt = (buffer) => {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const data = Buffer.concat([cipher.update(buffer), cipher.final()]);
  return Buffer.concat([iv, cipher.getAuthTag(), data]);
};

exports.decrypt = (payload) => {
  const iv = payload.subarray(0, 12);
  const tag = payload.subarray(12, 28);
  const data = payload.subarray(28);
  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  return Buffer.concat([decipher.update(data), decipher.final()]);
};
```

APPENDIX C – DATABASE



MongoDB Compass - 127.0.0.1:27017/encrypted_file_transfer.permissions

Connections Edit View Collection Help

Compass

{ } My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

- admin
- config
- efits_mobile_smoke
- encrypted_file_transfer
 - files
 - permissions**
 - transfer_logs
 - users
- local

permissions

127.0.0.1:27017 > encrypted_file_transfer > permissions

Documents 1 Aggregations Schema Indexes 5 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

25 1 - 1 of 1

```
{
  "_id": ObjectId('6abd41bb08720d84a65fcd9d'),
  "fileId": ObjectId('6abd41bb08720d84a65fcd99'),
  "fileName": "demo-secret.txt",
  "ownerId": ObjectId('6abd41bb08720d84a65fcd95'),
  "userId": ObjectId('6abd41bb08720d84a65fcd97'),
  "userEmail": "bob@example.com",
  "userName": "Bob Smith",
  "level": "download",
  "status": "active",
  "createdAt": ISODate('2026-09-30T17:07:07.280+08:00') September 30, 2026 at 5:07:07 PM UTC,
  "updatedAt": ISODate('2026-09-30T17:07:07.280+08:00') September 30, 2026 at 5:07:07 PM UTC,
  "__v": 0
}
```

MongoDB Compass - 127.0.0.1:27017/encrypted_file_transfer.files

Connections Edit View Collection Help

Compass

{ } My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

- admin
- config
- efits_mobile_smoke
- encrypted_file_transfer
 - files**
 - permissions
 - transfer_logs
 - users
- local

files

127.0.0.1:27017 > encrypted_file_transfer > files

Documents 4 Aggregations Schema Indexes 4 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

25 1 - 4 of 4

```
{
  "_id": ObjectId('6abd435b90aa4053b3d6412e'),
  "originalName": "secret.txt",
  "storedName": "e5b17e8e4075238ef22993d88d608459.enc",
  "ownerId": ObjectId('6abd41bb08720d84a65fcd95'),
  "ownerName": "Alice Johnson",
  "ownerEmail": "alice@example.com",
  "mimeType": "text/plain",
  "size": 21,
  "encryptedSize": 21,
  "sha256": "6e1d3926804c6d494146aef2562a386b6d22c209c33d40603424f96ecdde2db",
  "storagePath": "e5b17e8e4075238ef22993d88d608459.enc",
  "encryption": Object (7)
  createdAt: ISODate('2026-09-30T17:14:03.369+08:00') September 30, 2026 at 5:14:03 PM UTC,
  updatedAt: ISODate('2026-09-30T17:14:03.369+08:00') September 30, 2026 at 5:14:03 PM UTC,
  "__v": 0
}
```

```
{
  "_id": ObjectId('6abfa76d1a16df78a29b31ff'),
  "originalName": "Screenshot_2026-10-02-18-15-10-34_f73b71075b1de7323614b647fe394240.jpg",
  "storedName": "84c0ceee5897098c0850fea351ec7e5e.enc",
  "ownerId": ObjectId('6abfa73cb3988567fab08eac'),
  "ownerName": "Oliver",
  "ownerEmail": "oliver@gmail.com",
  "mimeType": "image/jpeg",
  "size": 309977
}
```

Welcome files +

127.0.0.1:27017 > encrypted_file_transfer > files Open MongoDB shell

Documents 4 Aggregations Schema Indexes 4 Validation

Type a query: { field: 'value' } or [Generate query](#) Explain Reset Find Options

25 1 - 4 of 4

```
{
  "_id": ObjectId('6abd41bb00720d84a65fcd99'),
  "originalName": "demo-secret.txt",
  "storedName": "22bd8dabf427d9b433690aac882829e3.enc",
  "ownerId": ObjectId('6abd41bb00720d84a65fcd95'),
  "ownerName": "Alice Johnson",
  "ownerEmail": "alice@example.com",
  "mimeType": "text/plain",
  "size": 88,
  "encryptedSize": 88,
  "sha256": "1fea8334620d2ec59b9a62a8ba03cbe43ec28913efbc419268d9f444a093202e",
  "storagePath": "22bd8dabf427d9b433690aac882829e3.enc",
  "encryption": Object (7),
  "createdAt": ISODate('2026-09-30T17:07:07.270+00:00') September 30, 2026 at 5:07:07 PM UTC,
  "updatedAt": ISODate('2026-09-30T17:07:07.270+00:00') September 30, 2026 at 5:07:07 PM UTC,
  "__v": 0
}
```

```
{
  "_id": ObjectId('6abd41bb00720d84a65fcd9b'),
  "originalName": "project-notes.md",
  "storedName": "6b83b0fe933796d2e95a49a2435a8ee6.enc",
  "ownerId": ObjectId('6abd41bb00720d84a65fcd95'),
  "ownerName": "Alice Johnson",
  "ownerEmail": "alice@example.com",
  "mimeType": "text/markdown",
  "size": 63
}
```

Welcome users +

127.0.0.1:27017 > efts_mobile_smoke > users Open MongoDB shell

Documents 7 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#) Explain Reset Find Options

25 1 - 7 of 7

```
{
  "_id": ObjectId('6abe0e61f404154c8caa6288'),
  "name": "Smoke A",
  "email": "smoke.a.1790840417219@example.com",
  "passwordHash": "$2a$10$w5XHp6bHy8KobW4oYeFatuZVcQUg05tbasgGunTcsDB9FYZBz580q",
  "role": "user",
  "active": true,
  "createdAt": ISODate('2026-10-01T07:40:17.426+00:00') October 1, 2026 at 7:40:17 AM UTC,
  "updatedAt": ISODate('2026-10-01T07:40:17.834+00:00') October 1, 2026 at 7:40:17 AM UTC,
  "__v": 0,
  "lastLoginAt": ISODate('2026-10-01T07:40:17.831+00:00') October 1, 2026 at 7:40:17 AM UTC
}
```

```
{
  "_id": ObjectId('6abe0e61f404154c8caa628b'),
  "name": "Smoke B",
  "email": "smoke.b.1790840417219@example.com",
  "passwordHash": "$2a$10$30CQu.RnyA1aknBJrDoFB0DIb4GavQ3tUF3X/lzEGPXN0/tLDKvIS",
  "role": "user",
  "active": true,
  "createdAt": ISODate('2026-10-01T07:40:17.673+00:00') October 1, 2026 at 7:40:17 AM UTC,
  "updatedAt": ISODate('2026-10-01T07:40:19.130+00:00') October 1, 2026 at 7:40:19 AM UTC,
  "__v": 0,
  "lastLoginAt": ISODate('2026-10-01T07:40:17.985+00:00') October 1, 2026 at 7:40:17 AM UTC
}
```

Welcome

users

+

127.0.0.1:27017 > efts_mobile_smoke > users

Documents 7

Aggregations

Schema

Indexes 2

Validation

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain

Reset

Find

Options ▶

25

1 - 7 of 7

◀

▶

⋮

{ }

⌂

```
_id: ObjectId('6abe04d6033c5578493576da')
name: "Smoke B"
email: "smoke.b.1790837973573@example.com"
passwordHash: "$2a$10$SGMtAeEsum/N/cKZ55QY4.tAzRxfqIjjAqM7KnFwXgfHhelcyYili"
role: "user"
active: true
createdAt: ISODate('2026-10-01T06:59:34.002+00:00') October 1, 2026 at 6:59:34 AM UTC
updatedAt: ISODate('2026-10-01T06:59:34.331+00:00') October 1, 2026 at 6:59:34 AM UTC
__v: 0
lastLoginAt: ISODate('2026-10-01T06:59:34.330+00:00') October 1, 2026 at 6:59:34 AM UTC
```

```
_id: ObjectId('6abe0e61f404154c8caa6288')
name: "Smoke A"
email: "smoke.a.1790840417219@example.com"
passwordHash: "$2a$10$Sw5XHp6bHy8KobW4oYeFatuZvcQUg05tbasGunTcsDB9FYZBz580q"
role: "user"
active: true
createdAt: ISODate('2026-10-01T07:40:17.426+00:00') October 1, 2026 at 7:40:17 AM UTC
updatedAt: ISODate('2026-10-01T07:40:17.834+00:00') October 1, 2026 at 7:40:17 AM UTC
__v: 0
lastLoginAt: ISODate('2026-10-01T07:40:17.831+00:00') October 1, 2026 at 7:40:17 AM UTC
```

Welcome

users

+

127.0.0.1:27017 > efts_mobile_smoke > users

Documents 7

Aggregations

Schema

Indexes 2

Validation

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain

Reset

Find

Options ▶

25

1 - 7 of 7

◀

▶

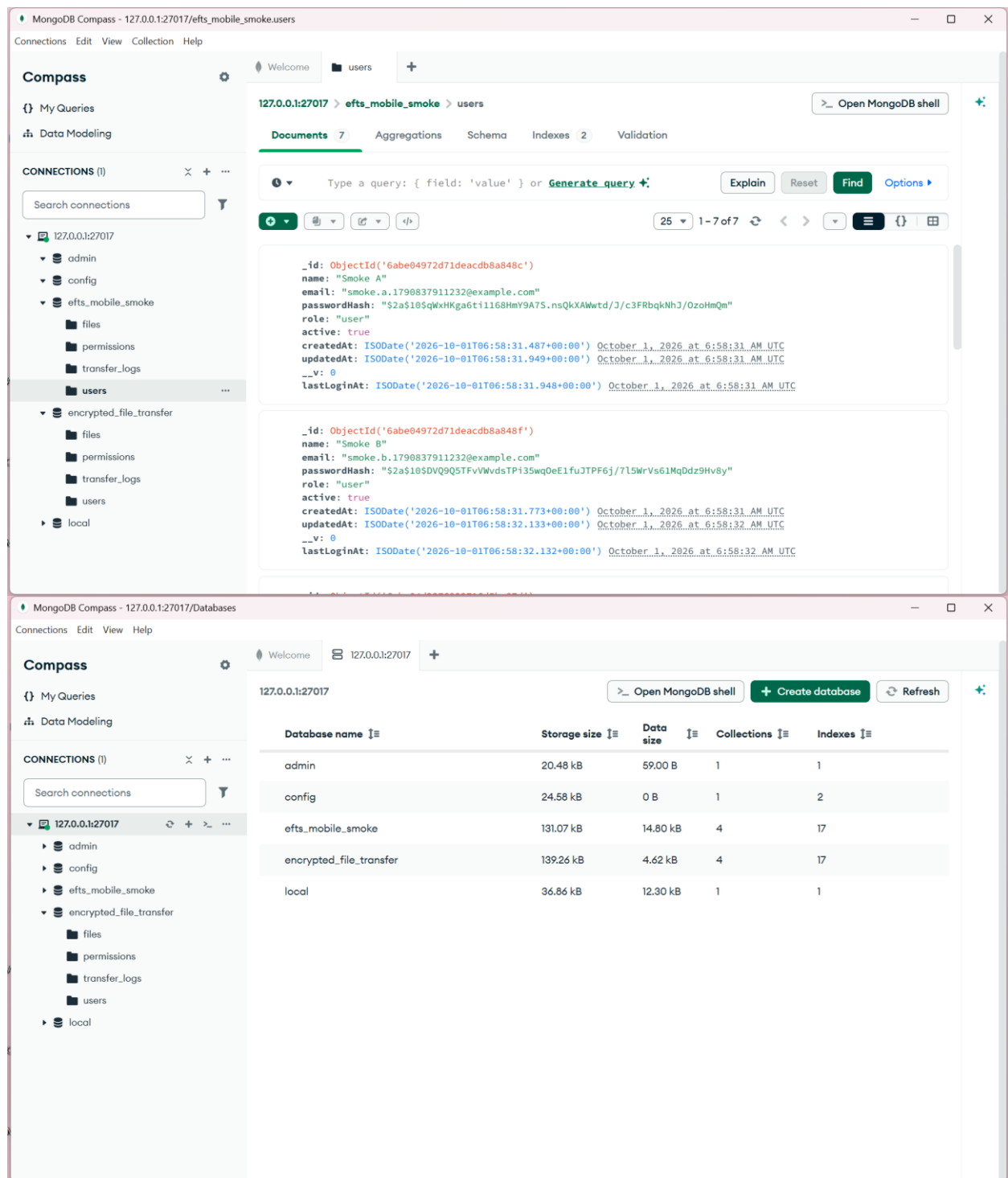
⋮

{ }

⌂

```
_id: ObjectId('6abe04d2376922716d5ba87d')
name: "Smoke Test Admin"
email: "admin@example.com"
passwordHash: "$2a$10$8goNRD6DXVqqb08GLwH8y04GoTF07ow4Gj5Yei1A7m34kfqT9KCoy"
role: "admin"
active: true
createdAt: ISODate('2026-10-01T06:59:30.457+00:00') October 1, 2026 at 6:59:30 AM UTC
updatedAt: ISODate('2026-10-01T07:40:19.000+00:00') October 1, 2026 at 7:40:19 AM UTC
__v: 0
lastLoginAt: ISODate('2026-10-01T07:40:18.999+00:00') October 1, 2026 at 7:40:18 AM UTC
```

```
_id: ObjectId('6abe04d5033c5578493576d7')
name: "Smoke A"
email: "smoke.a.1790837973573@example.com"
passwordHash: "$2a$10$RESUoLHJQnZV9TgAt4/DJ0GGCyA0GaYT324Yp7laBfn4at2807RWS"
role: "user"
active: true
createdAt: ISODate('2026-10-01T06:59:33.753+00:00') October 1, 2026 at 6:59:33 AM UTC
updatedAt: ISODate('2026-10-01T06:59:34.170+00:00') October 1, 2026 at 6:59:34 AM UTC
__v: 0
lastLoginAt: ISODate('2026-10-01T06:59:34.169+00:00') October 1, 2026 at 6:59:34 AM UTC
```



APPENDIX D – INSTALLATION AND SETUP

- Install Node.js (LTS), npm, Git, MongoDB, MongoDB Compass, Visual Studio Code, and Postman.
- Clone the project repository: `git clone <your_repository_url>`.
- Open the server folder and run `npm install`.
- Create a `.env` file with `PORT`, `MONGODB_URI=<your_mongodb_connection_string>`, `JWT_SECRET=<your_secret>`, and `ENCRYPTION_KEY=<your_encryption_key>`.
- Start MongoDB, then start the backend with `npm run dev` (or `npm start`).
- Open the client folder, run `npm install`, then `npm run dev` (or `npm start`).
- Open the application in a web browser at the address shown in the terminal.
- Import the Postman collection to test the API endpoints.

APPENDIX E – USER MANUAL

1. Registration

Open the registration page, enter your name, email, and a password, and submit. You will be told if the email is already registered.

2. Login

Enter your email and password on the login page. On success you are taken to the dashboard.

3. Uploading a File

On the dashboard choose Upload, select a file of an allowed type and size, and submit. The file is encrypted automatically.

4. Sharing / Access Permission

Open the permissions view for a file you own, select the user and permission level, and grant access. You can revoke access at any time.

5. Downloading a File

Select a file you own or that was shared with you and choose Download. If you lack permission, an error message is shown.

6. Logout

Choose Logout to end your session. You will need to log in again to access files.

APPENDIX F – GLOSSARY

Table H.1 – Glossary of Terms

Term	Meaning
API	A defined way for software components to communicate
REST	An architectural style using HTTP verbs and resource URLs
JWT	JSON Web Token, a signed token used for authentication
bcrypt	A one-way password hashing function
Encryption	Transforming data so it can be recovered only with the proper key
Hashing	One-way transformation that cannot be reversed to the original
Authentication	Verifying who a user is
Authorization	Determining what a user may do
ODM	Object Data Modelling, mapping application objects to database documents
ObjectId	MongoDB's default unique document identifier
BSON	Binary JSON, the format MongoDB uses to store documents
Middleware	Functions that run between receiving a request and sending a response
CORS	Browser rule controlling which origins may call an API
IV	Initialization vector, a per-encryption random value used by ciphers
Collection	A group of MongoDB documents, comparable in role to a table
Document	A single JSON-like record in a MongoDB collection

APPENDIX I – API REFERENCE (PROPOSED)

The following request and response outlines are proposed designs. Values are placeholders.

POST /api/auth/register

```
Request: { "name": "<name>", "email": "<email>", "password": "<password>" }
Success: 201 { "message": "Registered" }
Errors: 400 invalid input, 409 email already registered
```

POST /api/auth/login

```
Request: { "email": "<email>", "password": "<password>" }
Success: 200 { "token": "<jwt>" }
Errors: 401 invalid credentials
```

POST /api/files/upload

```
Headers: Authorization: Bearer <jwt>
Body: multipart/form-data, field 'file'
Success: 201 { "fileId": "<id>" }
Errors: 400 invalid file, 401 invalid token
```

GET /api/files

```
Headers: Authorization: Bearer <jwt>
Success: 200 [ { "_id": "<id>", "fileName": "<name>", "fileSize": <bytes> } ]
```

GET /api/files/:id/download

```
Headers: Authorization: Bearer <jwt>
Success: 200 file bytes (decrypted)
Errors: 401 invalid token, 403 access denied, 404 not found
```

POST /api/permissions

```
Headers: Authorization: Bearer <jwt> (file owner)
Request: { "fileId": "<id>", "userId": "<id>", "permission": "read" }
Success: 201
Errors: 403 not the owner
```

DELETE /api/permissions/:id

```
Headers: Authorization: Bearer <jwt> (file owner)
Success: 200
Errors: 403 not the owner, 404 not found
```

Geo Tag photos with guide:

